

Harmony - simulator design and use

Aleksandra Lekić, Haixiao Li, Saif Alsarayreh, Azadeh Kermansaravi,
Robert Dimitrovski

May 28, 2025

Contents

1	Introduction	2
2	AC/MTDC Optimal Power Flow	2
2.1	Pacakge Reuquirements	2
2.2	Preparing Data Files	2
2.3	Running a Simulation	7
2.4	Problem Modeling	7
2.4.1	Nomenclature	7
2.4.2	Original nonlinear formulation	9
2.4.3	Convex relaxed formulation	11
2.5	Cross-Language Implementation	14
2.5.1	Defining variables	14
2.5.2	Adding constraints	15
2.5.3	Establishing an objective	16
2.6	Accessing the Results	18
3	Mathematical Framework Implementation	20
3.1	Getting Start	20
3.1.1	Objectives	20
3.1.2	Implementation structure	20
3.2	Implementation and Computational Modeling of Power System Components	22
3.2.1	Impedance	23
3.2.2	Admittance	28
3.2.3	AC Source	31
3.2.4	Load	33
3.2.5	Transformer with magnetization resistance and inductance	35
3.2.6	Transformers with mutual inductance	37
3.2.7	Generator	45
3.2.8	Transmission Lines	48
3.2.9	Overhead line	51
3.2.10	Cable	56
3.2.11	MMC	59
3.2.12	Software Testing	66
3.3	Equivalent Impedance of the Subnetwork	68
3.3.1	Harmonic Resonance Modal Analysis Approach	68
3.3.2	Adjusted Modified Nodal Analysis Approach	68

1 Introduction

2 AC/MTDC Optimal Power Flow

The AC/MTDC optimal power flow (OPF) model presented in this manual is specifically designed for interconnected power systems consist of multiple AC power grids coupled with a single voltage source converter-based multi-terminal DC (VSC-MTDC) system, as illustrated in Fig. 2.1.

This section provides a concise guide on running codes, modifying input data, and accessing results. It serves as a basic introduction to help to get started with the code.

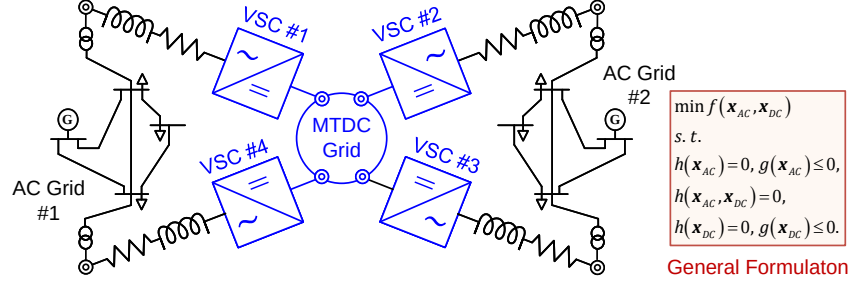


Figure 2.1: Overview of the AC/MTDC OPF problem

2.1 Package Requirements

The AC/MTDC optimal power flow (OPF) demo codes are developed using languages: [Matlab](#), [Julia](#), [Python](#), [C++](#). For each programming environment, we provide recommendations on the package versions used in our implementation to ensure consistency and reproducibility. While users may install later versions of these packages, it is advised to verify compatibility. [Gurobi](#) is used as a unified OPF solver for all language environments. As below, we outline the key dependencies.

- [Matlab r2021b](#) : `Yalmip 20230622`, `Matpower 7.1`
- [Julia v1.10](#) : `JuMP.jl 1.23` `Gurobi.jl 1.0.0` `CSV.jl 0.10.15` `DataFrames.jl 1.7.0`
- [Python v3.12](#) : `Pyomo.py 6.8.2` `Gurobipy.py 11.0` `Pandas.py 2.2.3` `Numpy.py 2.2.1`
- [C++](#) : `Gurobi C++ API` `Eigen 3.4.0`
- [Gurobi](#) : `Gurobi 9.5.2`

2.2 Preparing Data Files

The data required for AC/MTDC OPF includes the fundamental data of the ac power grid and the fundamental data of the dc power grid, which are listed and shown in Fig. 2.3. All data are stored in .CSV files due to its compatibility with various language environments.

Users can manually fill in power grid parameters into respective .CSV files based on their specific needs. Below we present the details regarding the respective .CSV. The style of the saved data in .CSV is basically consistent with `mpc` structure used in Matpower.

For the DC grid, the required fundamental data files includes `baseMW.dc.csv`, `pol.dc.csv`, `bus.dc.csv`, `branch.dc.csv`, `branch.dc.csv`. In the file, each column of the data corresponds to a parameter item. A specific rule needs to be followed when saving data into these files. The details about parameter items and descriptions of DC grid data files can be found in Table 1, 2, 3, 4, 5. Note that in `bus.dc.csv` and

Required data (.CSV)	
Power grids data sheets	
-baseMVA_ac	-baseMW_dc
-bus_ac	-pol_dc
-branch_ac	-bus_dc
-gen_ac	-branch_dc
-gencost_ac	-conv_dc

Figure 2.2: Required power grid data sheet

branch_dc.csv files, some parameter items may appear to be meaningless but serve as placeholders to maintain the same format style with the well-known Matpower.

For the AC grid, the required fundamental data files includes **baseMVA_ac.csv**, **bus_ac.csv**, **branch_ac.csv**, **gen_ac.csv**, **gencost_ac.csv**. The details about parameter items and descriptions of AC grid data files can be found in Table 6, 7, 8, 9, 10. Matpower provides various AC grid case files. Sometimes, users may want to directly select and integrate multiple AC grids without manually modifying data files. To facilitate this process, we provide auxiliary functions `merge_ac.m` and `save_csv.m`. User can build up [Matlab](#) script as below.

```

1 mpc_merged = merge_ac('case9', 'case14'); % merge_ac('case_a', 'case_b', '...')
2 save_csv(mpc_merged, directory); % If no saving path is provided, saved in the current directory.

```

After running the script, users can name the prefix of the series of .CSV filenames related to ac grids and directly type it. Finally, save information is shown in command window, like blow.

```

1 Enter a prefix of the CSV filenames: ac9ac14
2 Merged AC grid data has already been saved to:
3 D:\acdcopf\Tests\ac9ac14_bus.csv
4 D:\acdcopf\Tests\ac9ac14_gen.csv
5 D:\acdcopf\Tests\ac9ac14_branch.csv
6 D:\acdcopf\Tests\ac9ac14_gencost.csv

```

Column	Parameters	Description [Unit]
1	baseMW	Base power capacity [MW].

Table 1: baseMW_dc.csv file

Column	Parameters	Description [Unit]
1	pol	Number of poles (1=monopolar grid, 2=bipolar grid) [-].

Table 2: pol_dc.csv file

Column	Parameters	Description [Unit]
1	bus_i	Bus number [-].
2	—	N/A.
3	Pd	Active power loads [MW].
4	—	N/A.
5	—	N/A.
6	—	N/A.
7	—	N/A.
8	—	N/A.
9	—	N/A.
10	baseKV	Bus voltage level [kV].
11	—	N/A.
12	Vmax	Maximum bus voltage magnitude [p.u.].
13	Vmin	Minimum bus voltage magnitude [p.u.].

Table 3: bus_dc.csv file

Column	Parameters	Description [Unit]
1	fbus	From bus index [-].
2	tbus	To bus index [-].
3	r	Branch resistance [p.u.].
4	—	N/A.
5	—	N/A.
6	—	N/A.
7	—	N/A.
8	—	N/A.
9	—	N/A.
10	—	N/A.
11	—	N/A.
12	—	N/A.
13	—	N/A.

Table 4: branch_dc.csv file

Column	Parameters	Description [Unit]
1	busdc.i	Converter-connected DC bus index [-].
2	busac.i	Converter-connected AC bus index [-].
3	gridac	AC grid number of which converter belongs [-].
4	type_dc	Converter DC-side control modes (1=Constant DC power control, 2=Constant DC voltage control, 3=DC droop control) [-].
5	type_ac	Converter AC-side control modes (1=Constant AC voltage control, 2=Constant reactive power control) [-].
6	P_g	Setting of the active power output of converter DC terminal (applicable to the constant DC power control) [MW].
7	Q_g	Setting of the reactive power output of converter PCC bus (applicable to the constant reactive power control [MVar].
8	Vtar	Setting of the voltage amplitude of converter DC terminal (applicable to the constant DC voltage control) [p.u.].
9	rtf	Resistance of the converter transformer [p.u.].
10	xtf	Reactance of the converter transformer [p.u.].
11	bf	Conductance of the converter filter [p.u.].
12	rc	Resistance of the converter Phase reactor [p.u.].
13	xc	reactance of the converter Phase reactor [p.u.].
14	basekVac	Base voltage of the converter AC side [p.u.].
15	Vmmax	Maximum voltage magnitude of the converter AC side [p.u.].
16	Vmmin	Minimum voltage magnitude of the converter AC side [p.u.].
17	Imax	Maximum current magnitude of the converter [p.u.].
18	status	Converter status (1=in service, 0=out of service) [p.u.].
19	LossA	Constant loss coefficient [MW].
20	LossB	Linear loss coefficient [kV].
21	LossCR	Rectifier quadratic loss coefficient [Ω].
22	LossCI	Inverter quadratic loss coefficient [Ω].

Table 5: conv_dc.csv file

Column	Parameters	Description [Unit]
1	baseMVA	Base power capacity [MVA].

Table 6: baseMVA_ac.csv file

Column	Parameters	Description [Unit]
1	bus.i	Bus number [-].
2	type	Bus type (3=Slack, 2=PV, 1=PQ) [-]
3	Pd	Active power loads [MW].
4	Qd	Reactive power loads [MVar].
5	Gs	Shunt conductance (at V=1.0 p.u.) [MW].
6	Bs	Shunt susceptance (at V=1.0 p.u.) [MVar].
7	area	Area number [-].
8	Vm	Reactive power loads [p.u.].
9	Va	Reactive power loads [degrees].
10	baseKV	Bus voltage level [kV].
11	zone	Loss zone [-].
12	Vmax	Maximum bus voltage magnitude [p.u.].
13	Vmin	Minimum bus voltage magnitude [p.u.].

Table 7: bus_ac.csv file

Column	Parameters	Description [Unit]
1	fbus	From bus index [-].
2	tbus	To bus index [-].
3	r	Branch resistance [p.u.].
4	x	Branch reactance [p.u.].
5	b	Line charging susceptance.
6	rateA	Rate A (MVA limit for normal operation) [MVA].
7	rateB	Rate B (MVA limit for short-term) [MVA].
8	rateC	Rate C (MVA limit for emergency).
9	ratio	Transformer tap ratio (if applicable).
10	angle	Transformer phase shift angle [degrees].
11	status	Line status (1=active, 0=out-of-service).
12	angmin	Minimum voltage angle difference [-].
13	angmax	Maximum voltage angle difference [-].

Table 8: branch.ac.csv file

Column	Parameters	Description [Unit]
1	bus	Generator's bus number [-].
2	Pg	Active power output [MW].
3	Qg	Reactive power output [MVar].
4	Qmax	Maximum reactive power output [MVar].
5	Qmin	Minimum reactive power output [MVar].
6	Vg	Voltage magnitude setpoint [p.u.].
7	mBase	Total MVA base of generator [MVA].
8	status	Generator status (1=in service, 0=out of service) [-].
9	Pmax	Maximum active power output [MW].
10	Pmin	Minimum active power output [MW].
11	PC1	lower real power output of PQ capability curve [MW].
12	PC2	upper real power output of PQ capability curve [MW].
13	Qc1min	Minimum reactive power output at Pc1 [MVar].
14	Qc1max	Maximum reactive power at PC1 [MVar].
15	Qc2min	Minimum reactive power at PC2 [MVar].
16	Qc2max	Maximum reactive power at PC2 [MVar].
17	ramp_agc	Ramp rate for automatic generation control [MW/min].
18	ramp_10	Ramp rate for 10-minute reserves [MW].
19	ramp_30	Ramp rate for 30-minute reserves [MW].
20	ramp-q	Ramp rate for reactive power [MVar/min].
21	apf	Area participation factor [-].

Table 9: gen.ac.csv file

Column	Parameters	Description [Unit]
1	model	Cost function type(1=piecewise, 2=polynomial) [-].
2	startup	Startup cost [\$].
3	shutdown	Shutdown cost [\$].
4	n	Number of data points (if piecewise linear) or order of polynomial [\$].
5	cost	Coefficients of polynomial (from hgithest to lowest order) or piecewise data points.

Table 10: gencost.ac.csv file

2.3 Running a Simulation

The AC/MTDC OPF program in each language environment have core functions: `create_ac`-AC system data construction, `create_dc`-MTDC system data construction, `params_ac`-AC system parameter extraction, `params_dc`-DC system parameter extraction, and `main_acdcopf`-OPF model execution. Users should place all core functions in the same folder and then execute `main_acdcopf` to run AC/MTDC OPF. When running `main_acdcopf`, users need to provide the **prefix names** of the AC and MTDC system .csv files.

2.4 Problem Modeling

This section introduces the modeling of the AC/MTDC power system. The necessary constraint and objective functions to describe the optimal steady-state behavior for the AC/MTDC power system are introduced. Symbols appearing in subsequent formulas can be found below.

2.4.1 Nomenclature

Indexes and Sets

$(\bar{\cdot})/(\underline{\cdot})$ -The upper/lower bound of the corresponding optimization variables and parameters

$|\cdot|$ -Magnitude

$\mathcal{L}$ -Branch set

$\mathcal{N}$ -Node set

AC -Alternating current

$cost$ -Generation cost

$droop$ -Droop parameters

gen -Generator

h, i, j -System node

ij -System branch

$load$ -Load

$loss$ -Power loss

$MTDC$ -Multi-terminal direct current

ref -Reference value

VSC -Voltage source converter

Variables and Parameters Related to the AC Grid

θ_{ij}^{AC} -Phase difference for AC branch ij

$c_{i,2}^{AC}/c_{i,1}^{AC}/c_{i,0}^{AC}$ -Quadratic/Linear/Constant generation coefficient at AC bus i

c_{ij}^{AC}/s_{ij}^{AC} -Optimization variables linked with the AC nodal voltage

p_{ij}^{AC}/q_{ij}^{AC} -Real/Reactive power for AC branch ij

$p_{i,A2V}^{AC}/q_{i,A2V}^{AC}$ -Real/Reactive power delivered from the AC grid to the VSC station at AC bus i

$p_{i,gen}^{AC}/q_{i,gen}^{AC}$ -Real/Reactive power outputs of generator at AC bus i

$p_{i,load}^{AC}/q_{i,load}^{AC}$ -Real/Reactive power loads at AC bus i

p_i^{AC}/q_i^{AC} -Real/Reactive power injection at AC bus i

v_i^{AC} -AC bus i voltage

Variables and Parameters Related to the MTDC Grid

$k_{i,droop}^{MTDC}$ -Droop slope at MTDC bus i

l_{jh}^{MTDC} -Squared current of MTDC branch jh

$p_{i,ref}^{MTDC}$ -Power reference of DC power control/DC droop control at MTDC bus i

p_{jh}^{MTDC} -Power of MTDC branch jh

u_j^{MTDC} -Squared MTDC bus j voltage

v_j^{MTDC} -MTDC bus j voltage

$V_{i,ref}^{MTDC}$ -Voltage reference of DC voltage control/DC droop control at MTDC bus i

Variables and Parameters Related to the VSC Station

$\theta_s^{VSC}/\theta_f^{VSC}/\theta_c^{VSC}$ -VSC bus $s/f/c$ phase

$a_{loss}^{VSC}/b_{loss}^{VSC}/c_{loss}^{VSC}$ -Constant/Linear/Quadratic coefficient of VSC cpower loss function

b_f^{VSC} -Susceptance for VSC bus f to ground

$b_{sf}^{VSC}/b_{fc}^{VSC}$ -Susceptance for VSC branch sf/fc

$c_{sf}^{VSC}/s_{sf}^{VSC}/c_{cf}^{VSC}/s_{cf}^{VSC}$ -Optimization variables linked with the VSC nodal voltage

$g_{sf}^{VSC}/g_{fc}^{VSC}$ -Conductance for VSC branch sf/fc

I_c^{VSC} -Phase current at VSC bus c

l_c^{VSC} -Squared phase current at VSC bus c

p_{loss}^{VSC} -VSC power loss

$q_{s,ref}^{VSC}$ -Power reference of reactive power control at VSC bus s

$v_s^{VSC}/v_f^{VSC}/v_c^{VSC}$ -VSC bus $s/f/c$ voltage

$v_{s,ref}^{VSC}$ -Voltage reference of AC voltage control at VSC bus s

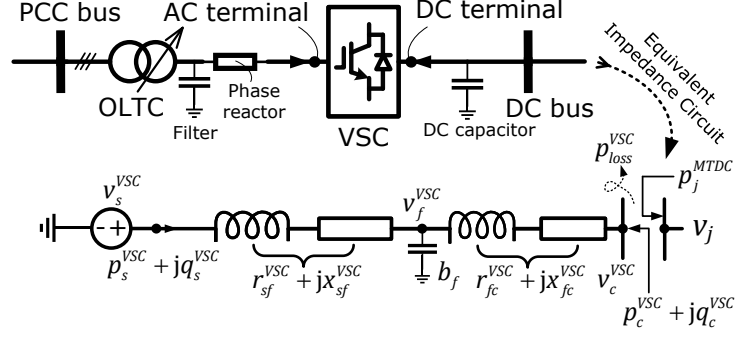


Figure 2.3: Equivalent impedance model of VSC

2.4.2 Original nonlinear formulation

The original AC/MTDC OPF formulation contains is formulated as a nonlinear programming (NLP) problem due to the presence of several nonlinear constraints, such as nonlinear power flow constraints and quadratic converter loss constraints. The explicit formulation of the original AC/MTDC OPF model as below.

Original AC/MTDC OPF Formulation	[NLP]
----------------------------------	-------

Variables:

$p_i^{AC}, q_i^{AC}, p_{i,gen}^{AC}, q_{i,gen}^{AC}, p_{i,A2V}^{AC}, q_{i,A2V}^{AC}, \theta_i^{AC}, v_i^{AC}$	$i \in \mathcal{N}^{AC}$
p_{ij}^{AC}, q_{ij}^{AC}	$(i, j) \in \mathcal{L}^{AC}$
p_j^{MTDC}, v_j^{MTDC}	$j \in \mathcal{N}^{MTDC}$
p_{jh}^{MTDC}	$(j, h) \in \mathcal{L}^{MTDC}$
$p_s^{VSC}, q_s^{VSC}, \theta_s^{VSC}, v_s^{VSC}$	$s \in \mathcal{N}^{VSC}$
$p_c^{VSC}, q_c^{VSC}, \theta_c^{VSC}, v_c^{VSC}, I_c^{VSC}$	$c \in \mathcal{N}^{VSC}$
$q_f^{VSC}, \theta_f^{VSC}, v_f^{VSC}$	$f \in \mathcal{N}^{VSC}$

AC constraints:

$$p_{ij}^{AC} = g_{ij}^{AC} |v_i^{AC}|^2 - |v_i^{AC}| |v_j^{AC}| (g_{ij}^{AC} \cos \theta_{ij}^{AC} + b_{ij}^{AC} \sin \theta_{ij}^{AC}), \quad i, j \in \mathcal{N}^{AC}, \quad (i, j) \in \mathcal{L}^{AC} \quad (2.1a)$$

$$q_{ij}^{AC} = -b_{ij}^{AC} |v_i^{AC}|^2 + |v_i^{AC}| |v_j^{AC}| (b_{ij}^{AC} \cos \theta_{ij}^{AC} - g_{ij}^{AC} \sin \theta_{ij}^{AC}), \quad i, j \in \mathcal{N}^{AC}, \quad (i, j) \in \mathcal{L}^{AC} \quad (2.1b)$$

$$p_i^{AC} = \sum_{(i,j)} p_{ij}^{AC} + \left(\sum_j G_{ij}^{AC} \right) (v_i^{AC})^2, \quad i, j \in \mathcal{N}^{AC}, \quad (i, j) \in \mathcal{L}^{AC} \quad (2.1c)$$

$$q_i^{AC} = \sum_{(i,j)} q_{ij}^{AC} + \left(\sum_j -B_{ij}^{AC} \right) (v_i^{AC})^2, \quad i, j \in \mathcal{N}^{AC}, \quad (i, j) \in \mathcal{L}^{AC} \quad (2.1d)$$

$$p_i^{AC} = p_{i,gen}^{AC} - p_{i,load}^{AC} - p_{i,A2V}^{AC}, \quad i \in \mathcal{N}^{AC} \quad (2.2a)$$

$$q_i^{AC} = q_{i,gen}^{AC} - q_{i,load}^{AC} - q_{i,A2V}^{AC}, \quad i \in \mathcal{N}^{AC} \quad (2.2b)$$

$$\underline{p}_{i,gen}^{AC} \leq p_{i,gen}^{AC} \leq \bar{p}_{i,gen}^{AC}, \quad i \in \mathcal{N}^{AC} \quad (2.3a)$$

$$\underline{q}_{i,gen}^{AC} \leq q_{i,gen}^{AC} \leq \bar{q}_{i,gen}^{AC}, \quad i \in \mathcal{N}^{AC} \quad (2.3b)$$

$$\underline{|v_i^{AC}|} \leq |v_i^{AC}| \leq \overline{|v_i^{AC}|}, \quad i \in \mathcal{N}^{AC} \quad (2.4)$$

MTDC constraints:

$$p_{jh}^{MTDC} = \rho^{MTDC} \cdot v_j^{MTDC} (v_j^{MTDC} - v_h^{MTDC}) y_{jh}^{MTDC}, \quad j, h \in \mathcal{N}^{MTDC}, \quad (j, h) \in \mathcal{L}^{MTDC} \quad (2.5a)$$

$$p_j^{MTDC} = \sum_{(j,h)} p_{jh}^{MTDC}, \quad j, h \in \mathcal{N}^{MTDC}, \quad (j, h) \in \mathcal{L}^{MTDC} \quad (2.5b)$$

$$v_j^{MTDC} \leq v_j^{MTDC} \leq \bar{v}_j^{MTDC}, \quad j \in \mathcal{N}^{MTDC} \quad (2.6)$$

$$p_j^{MTDC} = p_{j,ref}^{MTDC}, \quad j \in \mathcal{N}^{MTDC} \quad (2.7a)$$

$$v_j^{MTDC} = v_{j,ref}^{MTDC}, \quad j \in \mathcal{N}^{MTDC} \quad (2.7b)$$

$$p_j^{MTDC} = p_{j,ref}^{MTDC} - \frac{1}{k_{j,droop}^{MTDC}} (v_j^{MTDC} - v_{j,ref}^{MTDC}), \quad j \in \mathcal{N}^{MTDC} \quad (2.7c)$$

VSC constraints:

$$p_s^{VSC} = p_{i,A2V}^{AC}, \quad s \in \mathcal{N}^{VSC}, \quad i \in \mathcal{N}^{AC} \quad (2.8a)$$

$$q_s^{VSC} = q_{i,A2V}^{AC}, \quad s \in \mathcal{N}^{VSC}, \quad i \in \mathcal{N}^{AC} \quad (2.8b)$$

$$p_s^{VSC} = |v_s^{VSC}|^2 g_{sf}^{VSC} - |v_s^{VSC}| |v_f^{VSC}| [g_{sf}^{VSC} \cos(\theta_s^{VSC} - \theta_f^{VSC}) + b_{sf}^{VSC} \sin(\theta_s^{VSC} - \theta_f^{VSC})], \quad s, f \in \mathcal{N}^{VSC} \quad (2.9a)$$

$$q_s^{VSC} = -|v_s^{VSC}|^2 b_{sf}^{VSC} - |v_s^{VSC}| |v_f^{VSC}| [g_{sf}^{VSC} \sin(\theta_s^{VSC} - \theta_f^{VSC}) - b_{sf}^{VSC} \cos(\theta_s^{VSC} - \theta_f^{VSC})], \quad s, f \in \mathcal{N}^{VSC} \quad (2.9b)$$

$$p_c^{VSC} = |v_c^{VSC}|^2 g_{fc}^{VSC} + |v_f^{VSC}| |v_c^{VSC}| [g_{fc}^{VSC} \cos(\theta_f^{VSC} - \theta_c^{VSC}) - b_{fc}^{VSC} \sin(\theta_f^{VSC} - \theta_c^{VSC})], \quad f, c \in \mathcal{N}^{VSC} \quad (2.10a)$$

$$q_c^{VSC} = -|v_c^{VSC}|^2 b_{fc}^{VSC} + |v_f^{VSC}| |v_c^{VSC}| [g_{fc}^{VSC} \sin(\theta_f^{VSC} - \theta_c^{VSC}) + b_{fc}^{VSC} \cos(\theta_f^{VSC} - \theta_c^{VSC})], \quad f, c \in \mathcal{N}^{VSC} \quad (2.10b)$$

$$q_f^{VSC} = -|v_f^{VSC}|^2 b_f^{VSC}, \quad f \in \mathcal{N}^{VSC} \quad (2.11)$$

$$p_c^{VSC} + p_{loss}^{VSC} + p_j^{MTDC} = 0, \quad j \in \mathcal{N}^{MTDC}, \quad c \in \mathcal{N}^{VSC} \quad (2.12)$$

$$p_{loss}^{VSC} = a_{loss}^{VSC} + b_{loss}^{VSC} |I_c^{VSC}| + c_{loss}^{VSC} |I_c^{VSC}|^2, \quad c \in \mathcal{N}^{VSC} \quad (2.13a)$$

$$0 \leq |I_c^{VSC}| \leq \overline{|I_c^{VSC}|}, \quad c \in \mathcal{N}^{VSC} \quad (2.13b)$$

$$(v_c^{VSC})^2 \cdot |I_c^{VSC}|^2 = (p_c^{VSC})^2 + (q_c^{VSC})^2, \quad c \in \mathcal{N}^{VSC} \quad (2.14)$$

$$v_s^{VSC} = v_{s,ref}^{VSC}, \quad s \in \mathcal{N}^{VSC} \quad (2.15a)$$

$$q_s^{VSC} = q_{s,ref}^{VSC}, \quad s \in \mathcal{N}^{VSC} \quad (2.15b)$$

Goal:

$$\min \sum_i c_{i,2}^{AC} (p_{i,gen}^{AC})^2 + c_{i,1}^{AC} p_{i,gen}^{AC} + c_{i,0}^{AC}, \quad i \in \mathcal{N}^{AC} \quad (2.16)$$

-
- **AC Constraints:** ✓(2.1a)-(2.1d) are power flow constraints. ✓(2.2a)-(2.2b) are nodal power balance equations. ✓(2.3a)-(2.3b) are generator power output limits. (2.4) is the voltage magnitude limit.
 - **MTDC Constraints:** ✓(2.5a)-(2.5b) are power flow constraints. ✓(2.6) is the voltage magnitude limit. ✓(2.7a)-(2.7c) represents the VSC control mode at DC terminal. Note that only one of them is hold depending on the chosen control model.
 - **VSC Constraint:** ✓(2.8a)-(2.8b) are the active and reactive power couplings between the AC grid and VSC. ✓(2.9a)-(2.9b) are the active and reactive power injections from the view of node s . ✓(2.10a)-(2.10b) are the active and reactive power injections from the view of node c . ✓(2.11) is the reactive power injection from the view of node f . If we consider the latest converters with the MMC topology, the filter is not required to install at node f and $b_f^{VSC} = 0$. ✓(2.12) is the active power coupling between the MTDC grid and VSC. ✓(2.13a)-(2.13b) are related to the converter loss function. ✓(2.14) denote the voltage and current relationship from the view of node c . ✓(2.15a)-(2.15b) represent the VSC control mode at AC PCC. Note that only one of them holds depending on the chosen control model.
 - **Goal:** ✓(2.16) represent minimizing power generation costs.

2.4.3 Convex relaxed formulation

The original AC/DC OPF problem is NLP. Such a problem is not favored due to their limited scalability and computational challenges. Give these reasons, we formulate a convex relaxed version of the original AC/DC OPF problem. Our demo code is based on the convex relaxed version as below.

Variables:

$$\begin{array}{ll}
p_i^{AC}, q_i^{AC}, p_{i,gen}^{AC}, q_{i,gen}^{AC}, p_{i,A2V}^{AC}, q_{i,A2V}^{AC}, s_{ij}^{AC}, c_{ij}^{AC} & i, j \in \mathcal{N}^{AC} \\
p_{ij}^{AC}, q_{ij}^{AC} & (i, j) \in \mathcal{L}^{AC} \\
p_j^{MTDC}, v_j^{MTDC} & j \in \mathcal{N}^{MTDC} \\
p_{jh}^{MTDC}, l_{jh}^{MTDC} & (j, h) \in \mathcal{L}^{MTDC} \\
p_s^{VSC}, q_s^{VSC} & s \in \mathcal{N}^{MTDC} \\
p_c^{VSC}, q_c^{VSC}, I_c^{VSC}, l_c^{VSC} & c \in \mathcal{N}^{MTDC} \\
q_f^{VSC} & f \in \mathcal{N}^{MTDC} \\
c_{mn}^{VSC}, c_{mn}^{VSC} & m, n \in \{s, f, c\} \quad s, f, c \in \mathcal{N}^{MTDC}
\end{array}$$

AC Constraints:

$$p_{ij}^{AC} = G_{ij} (c_{ii}^{AC} - c_{ij}^{AC}) + B_{ij} s_{ij}^{AC}, \quad i, j \in \mathcal{N}^{AC}, \quad (i, j) \in \mathcal{L}^{AC} \quad (2.17a)$$

$$q_{ij}^{AC} = -B_{ij} (c_{ii}^{AC} - c_{ij}^{AC}) + G_{ij} s_{ij}^{AC}, \quad i, j \in \mathcal{N}^{AC}, \quad (i, j) \in \mathcal{L}^{AC} \quad (2.17b)$$

$$p_i^{AC} = c_{ii}^{AC} G_{ii} + \sum_j (c_{ij}^{AC} G_{ij} - s_{ij}^{AC} B_{ij}), \quad i, j \in \mathcal{N}^{AC} \quad (2.17c)$$

$$q_i^{AC} = -c_{ii}^{AC} B_{ii} - \sum_j (c_{ij}^{AC} B_{ij} + s_{ij}^{AC} G_{ij}), \quad i, j \in \mathcal{N}^{AC} \quad (2.17d)$$

$$c_{ij}^{AC} = c_{ji}^{AC}, \quad i, j \in \mathcal{N}^{AC} \quad (2.17e)$$

$$s_{ij}^{AC} = -s_{ji}^{AC}, \quad i, j \in \mathcal{N}^{AC} \quad (2.17f)$$

$$(c_{ij}^{AC})^2 + (s_{ij}^{AC})^2 \leq c_{ii}^{AC} c_{jj}^{AC}, \quad i, j \in \mathcal{N}^{AC} \quad (2.17g)$$

$$p_i^{AC} = p_{i,gen}^{AC} - p_{i,load}^{AC} - p_{i,A2V}^{AC}, \quad i \in \mathcal{N}^{AC} \quad (2.18a)$$

$$q_i^{AC} = q_{i,gen}^{AC} - q_{i,load}^{AC} - q_{i,A2V}^{AC}, \quad i \in \mathcal{N}^{AC} \quad (2.18b)$$

$$\underline{p}_{i,gen}^{AC} \leq p_{i,gen}^{AC} \leq \bar{p}_{i,gen}^{AC}, \quad i \in \mathcal{N}^{AC} \quad (2.19a)$$

$$\underline{q}_{i,gen}^{AC} \leq q_{i,gen}^{AC} \leq \bar{q}_{i,gen}^{AC}, \quad i \in \mathcal{N}^{AC} \quad (2.19b)$$

$$|v_i^{AC}|^2 \leq c_{ii}^{AC} \leq |v_i^{AC}|^2, \quad i \in \mathcal{N}^{AC} \quad (2.20)$$

MTDC constraints:

$$p_j^{MTDC} = \sum_{(j,h)} p_{jh}^{MTDC}, \quad j \in \mathcal{N}^{MTDC}, \quad (j, h) \in \mathcal{L}^{MTDC} \quad (2.21a)$$

$$p_{jh}^{MTDC} + p_{hj}^{MTDC} = r_{jh}^{MTDC} l_{jh}^{MTDC}, \quad (j, h) \in \mathcal{L}^{MTDC} \quad (2.21b)$$

$$u_j^{MTDC} - u_h^{MTDC} = r_{jh}^{MTDC} (p_{jh}^{MTDC} - p_{hj}^{MTDC}), \quad j, h \in \mathcal{N}^{MTDC}, \quad (j, h) \in \mathcal{L}^{MTDC} \quad (2.21c)$$

$$p_{jh}^{MTDC} \leq l_{jh}^{MTDC} u_j^{MTDC} \quad j \in \mathcal{N}^{MTDC}, \quad (j, h) \in \mathcal{L}^{MTDC} \quad (2.21d)$$

$$\underline{u}_j^{MTDC} \leq u_j^{MTDC} \leq \bar{u}_j^{MTDC}, \quad j \in \mathcal{N}^{MTDC} \quad (2.22)$$

$$p_j^{MTDC} = p_{j,ref}^{MTDC}, \quad j \in \mathcal{N}^{MTDC} \quad (2.23a)$$

$$u_j^{MTDC} = (v_{j,ref}^{MTDC})^2, \quad j \in \mathcal{N}^{MTDC} \quad (2.23b)$$

$$p_j^{MTDC} = p_{j,ref}^{MTDC} - \frac{1}{k_{j,droop}^{MTDC}} \left(\frac{1}{2} + \frac{1}{2} u_j^{MTDC} - v_{j,ref}^{MTDC} \right), \quad j \in \mathcal{N}^{MTDC} \quad (2.23c)$$

VSC constraints:

$$p_s^{VSC} = p_{i,A2V}^{AC}, \quad s \in \mathcal{N}^{VSC} \quad (2.24a)$$

$$q_s^{VSC} = q_{i,A2V}^{AC}, \quad s \in \mathcal{N}^{VSC} \quad (2.24b)$$

$$p_s^{VSC} = c_{ss}^{VSC} g_{sf}^{VSC} - c_{sf}^{VSC} g_{sf}^{VSC} + s_{sf}^{VSC} b_{sf}^{VSC}, \quad s, f \in \mathcal{N}^{MTDC} \quad (2.25a)$$

$$q_s^{VSC} = -c_{ss}^{VSC} b_{sf}^{VSC} + c_{sf}^{VSC} b_{sf}^{VSC} + s_{sf}^{VSC} g_{sf}^{VSC}, \quad s, f \in \mathcal{N}^{MTDC} \quad (2.25b)$$

$$c_{sf}^{VSC} = c_{fs}^{VSC}, \quad s, f \in \mathcal{N}^{AC} \quad (2.25c)$$

$$s_{sf}^{VSC} = -s_{fs}^{VSC}, \quad s, f \in \mathcal{N}^{VSC} \quad (2.25d)$$

$$(c_{sf}^{VSC})^2 + (s_{sf}^{VSC})^2 \leq c_{ss}^{VSC} c_{ff}^{VSC}, \quad s, f \in \mathcal{N}^{AC} \quad (2.25e)$$

$$p_c^{VSC} = c_{cc}^{VSC} g_{cf}^{VSC} - c_{cf}^{VSC} g_{cf}^{VSC} + s_{cf}^{VSC} b_{cf}^{VSC}, \quad c, f \in \mathcal{N}^{MTDC} \quad (2.26a)$$

$$q_c^{VSC} = -c_{cc}^{VSC} b_{cf}^{VSC} + c_{cf}^{VSC} b_{cf}^{VSC} + s_{cf}^{VSC} g_{cf}^{VSC}, \quad c, f \in \mathcal{N}^{MTDC} \quad (2.26b)$$

$$c_{cf}^{VSC} = c_{fc}^{VSC}, \quad c, f \in \mathcal{N}^{VSC} \quad (2.26c)$$

$$s_{cf}^{VSC} = -s_{fc}^{VSC}, \quad c, f \in \mathcal{N}^{VSC} \quad (2.26d)$$

$$(c_{cf}^{VSC})^2 + (s_{cf}^{VSC})^2 \leq c_{cc}^{VSC} c_{ff}^{VSC}, \quad c, f \in \mathcal{N}^{VSC} \quad (2.26e)$$

$$q_f^{VSC} = -c_{ff}^{VSC} b_f^{VSC}, \quad f \in \mathcal{N}^{VSC} \quad (2.27)$$

$$p_c^{VSC} + p_{loss}^{VSC} + p_j^{MTDC} = 0, \quad j \in \mathcal{N}^{MTDC}, \quad c \in \mathcal{N}^{VSC} \quad (2.28)$$

$$p_{loss}^{VSC} = a_{loss}^{VSC} + b_{loss}^{VSC} |I_c^{VSC}| + c_{loss}^{VSC} l_c^{VSC}, \quad c \in \mathcal{N}^{VSC} \quad (2.29a)$$

$$0 \leq |I_c^{VSC}| \leq \left| \overline{I_c^{VSC}} \right|, \quad c \in \mathcal{N}^{VSC} \quad (2.29b)$$

$$0 \leq l_c^{VSC} \leq \left| \overline{I_c^{VSC}} \right|^2, \quad c \in \mathcal{N}^{VSC} \quad (2.29c)$$

$$|I_c^{VSC}|^2 \leq l_c^{VSC}, \quad c \in \mathcal{N}^{VSC} \quad (2.30a)$$

$$(p_c^{VSC})^2 + (q_c^{VSC})^2 \leq c_{cc}^{VSC} \cdot l_c^{VSC}, \quad c \in \mathcal{N}^{VSC} \quad (2.30b)$$

$$c_{ss}^{VSC} = (v_{s,ref}^{VSC})^2, \quad s \in \mathcal{N}^{VSC} \quad (2.31a)$$

$$q_s^{VSC} = q_{s,ref}^{VSC}, \quad s \in \mathcal{N}^{VSC} \quad (2.31b)$$

Goal:

$$\min \sum_i c_{i,2}^{AC} (p_{i,gen}^{AC})^2 + c_{i,1}^{AC} p_{i,gen}^{AC} + c_{i,0}^{AC}, \quad i \in \mathcal{N}^{AC} \quad (2.32)$$

-
- **AC Constraints:** ✓(2.17a)-(2.17g) consist of the second-order cone (SOC) relaxed power flow constraints [1]. ✓(2.18a)-(2.18b) are nodal power balance equations. ✓(2.19a)-(2.19b) are generator power output limits. (2.20) is the squared voltage magnitude limit.
 - **MTDC Constraints:** ✓(2.21a)-(2.21d) consist of the SOC relaxed power flow constraints [2]. ✓(2.22) is the squared voltage magnitude limit. ✓(2.23a)-(2.23c) represents the VSC control mode at DC terminal. Especially, the term v_{ij}^{MTDC} is replaced by the expression with respect to u_{ij}^{MTDC} in (2.23c), through Talyor expansion. Note that for (2.23a)-(2.23c), only one of them holds depending on the chosen control model.
 - **VSC Constraint:** ✓(2.24)-(2.24b) are the active and reactive power couplings between the AC grid and VSC. ✓(2.25a)-(2.25e) consist of the SOC relaxed active and reactive power injections from the view of node s . ✓(2.26a)-(2.26e) consist of the SOC relaxed active and reactive power injections from the view of node c . ✓(2.27) is the reactive power injection from the view of node f . ✓(2.28) is the active power coupling between the MTDC grid and VSC. ✓(2.29a)-(2.29c) are related to the converter loss function. ✓(2.30a)-(2.30b) consist of the SOC relaxed voltage and current relationship from the view of node c . ✓(2.31a)-(2.31b) represent the VSC control mode at AC PCC. Note that only one of them is hold depending on the chosen control model.

- **Goal:** ✓(2.32) represent minimizing power generation costs.

2.5 Cross-Language Implementation

To help users better understand the above OPF modeling in different language environments, we show some representative syntax below. The involved defining variables, adding constraints, setting optimization objectives, all these steps are implemented in the main function, referred to as `main_acdcopf`.

2.5.1 Defining variables

In [Matlab](#), Yalmip package is used. Under such a framework, `sdpvar` is used to define the required variables for AC/MTDC OPF, like below

```
1 var_dc = sdpvar(2*nbuses_dc+8*nconvs_dc, 1);
2 ...
3 % 1. vn2_dc -the square of dc nodal voltage
4 vn2_dc = var_dc(1:nbuses_dc, 1);
```

In [Matlab](#), to handle three-dimensional variables where each third-dimension index (k) has different (i, j) dimensions, `cell` is used to store multiple two-dimensional variables, effectively representing a three-dimensional data structure. We apply this rule in defining the variables related to multiple AC power grids, like below:

```
1 pij_ac = cell(ngrids, 1)
2 var_ac{ng} = sdpvar(3*nbuses_ac{ng}+2*ngens_ac{ng}, 1);
3 ...
4 % 6. pij_ac -branch active power flow
5 pij_ac{ng} = sdpvar(nbuses_ac{ng}, nbuses_ac{ng}, 'full');
```

In [Julia](#), JuMP package is used. Under such a framework, we start with `model = direct_model()`, then define variables like below

```
1 var_size_dc = 2*nbuses_dc + 8*nconvs
2 @variable(model, var_dc[1:var_size_dc])
3 ...
4 # 1. vn2_dc -the square of dc nodal voltage
5 vn2_dc = var_dc[1:nbuses_dc]
```

In [Julia](#), to address the issue of defining three-dimensional variables related to multiple AC grids, we use `Vector` to save two-dimension variables like below

```
1 pij_ac = Vector{Matrix{VariableRef}}(undef, ngrids)
2 var_size_ac = 3*nbuses_ac[ng] + 2*ngens_ac[ng]
3 var_ac[ng] = @variable(model, [1:var_size_ac])
4 ...
5 # 6. pij_ac -branch active power flow
6 pij_ac[ng] = @variable(model, [1:nbuses_ac[ng], 1:nbuses_ac[ng]])
```

In [Python](#), Pyomo package is used. Under such a framework, we start with `model = ConcreteModel()`, then we define variables like below

```
1 # 1. vn2_dc -the square of dc nodal voltage
2 model.vn2_dc = Var(range(nbuses_dc), domain=NonNegativeReals)
```

In [Python](#), to address the issue of defining three-dimensional variables related to multiple AC grids, we directly define like below

```
1 # 6. pij_ac -branch active power flow
2 model.pij_ac = Var(
3     [(ng, i, j) for ng in range(ngrids) for i in range(nbuses_ac[ng]) for j in range(nbuses_ac[ng])],
4     domain=Reals
5 )
```

In [C++](#), Gurobi C++ API is used. Under such a framework, the blow code snippet is required at start

```
1 GRBEnv env = GRBEnv(true);
2 env.start();
3 GRBModel model = GRBModel(env);
```

`model.addVar` is used to define the required variables, like below

```
1 // 1. vn2_dc: the square of DC nodal voltage
2 Eigen::Matrix<GRBVar, Eigen::Dynamic, 1> vn2_dc(nbuses_dc);
3 for (int i = 0; i < nbuses_dc; ++i) {
4     vn2_dc(i) = model.addVar(-GRB_INFINITY, GRB_INFINITY, 0.0, GRB_CONTINUOUS);
5 }
```

In [C++](#), to address the issue of defining three-dimensional variables related to multiple AC grids, we use `Eigen::Matrix` to save two-dimension variables like below

```
1 // 6. pij_ac -branch active power flow
2 pij_ac[ng] = Eigen::Matrix<GRBVar, Eigen::Dynamic, Eigen::Dynamic>(nbuses_ac[ng], nbuses_ac[ng]);
3 for (int i = 0; i < nbuses_ac[ng]; ++i) {
4     for (int j = 0; j < nbuses_ac[ng]; ++j) {
5         pij_ac[ng](i, j) = model.addVar(-GRB_INFINITY, GRB_INFINITY, 0.0, GRB_CONTINUOUS);
6     }
7 }
```

2.5.2 Adding constraints

In [Matlab](#), constraints are strongly recommended to be written in matrix formulations, as [Matlab](#) is highly optimized for matrix operations. Yalmip allows vectorized operations, enabling constraints to be defined in a compact and computationally efficient manner. For example, the soc relaxed MTDC power flow constraints (2.22a)-(2.22d) can be written in the compact matrix form below.

```
1 % 1. constraints for dc power flow -second-order cone relaxation
2 zij_dc = 1./abs(ybus_dc); zij_dc = zij_dc-diag(diag(zij_dc));
3 zij_dc(isinf(zij_dc)) = 1e4;
4 con_dc = [con_dc; pn_dc == sum(pij_dc, 2) * pol_dc];
5 con_dc = [con_dc; pij_dc + pij_dc' == zij_dc .* lij_dc];
6 con_dc = [con_dc; repmat(vn2_dc, 1, nbuses_dc) - repmat(vn2_dc', nbuses_dc, 1) == zij_dc .* (pij_dc - pij_dc')
7 ];
8 con_dc = [con_dc; pij_dc.^2 <= lij_dc .* repmat(vn2_dc, 1, nbuses_dc)];
9 con_dc = [con_dc; vn2_dc >= 0; lij_dc >= 0];
```

In [Julia](#), JuMP supports vectorized matrix operations and allows constraints to be written compactly. For efficient computation, matrix-based constraints are also recommended, and (2.22a)-(2.22d) can be written below

```
1 # 1. constraints for dc power flow -second-order cone relaxation
2 zij_dc = 1.0 ./ abs.(ybus_dc)
3 zij_dc = zij_dc .- Diagonal(diagm(0 => diag(zij_dc)))
4 zij_dc[isinf.(zij_dc)] .= 1e4
5 @constraint(model, pn_dc == pol_dc * sum(pij_dc, dims=2))
6 @constraint(model, pij_dc .+ pij_dc' == zij_dc .* lij_dc)
7 @constraint(model, vn2_dc .- vn2_dc' == zij_dc .* (pij_dc .- pij_dc'))
8 @constraint(model, pij_dc .^ 2 .<= lij_dc .* vn2_dc)
9 @constraint(model, 0 .<= vn2_dc)
10 @constraint(model, 0 .<= lij_dc)
```

In [Python](#), Pyomo does not support matrix operations. In this case, constraints like (2.22a)-(2.22d) must be defined using loops like below

```
1 # 1. constraints for dc power flow -second-order cone relaxation
2 zij_dc = 1.0 / np.abs(ybus_dc.toarray())
3 np.fill_diagonal(zij_dc, 0)
4 zij_dc[np.isinf(zij_dc)] = 1e4
5
```

```

6 def nodal_power_flow_dc(model, i):
7     return model.pn_dc[i] == pol_dc * sum(model.pij_dc[i, :])
8 model.nodal_power_flow_dc = Constraint(range(nbuses_dc), rule = nodal_power_flow_dc)
9
10 def branch_power_flow_dc(model, i, j):
11     return model.pij_dc[i, j] + model.pij_dc[j, i] == zij_dc[i, j] * model.lij_dc[i, j]
12 model.branch_power_flow_dc = Constraint(range(nbuses_dc), range(nbuses_dc), rule = branch_power_flow_dc)
13
14 def voltage_diff_dc(model, i, j):
15     return model.vn2_dc[i] - model.vn2_dc[j] == zij_dc[i, j] * (model.pij_dc[i, j] - model.pij_dc[j, i])
16 model.voltage_diff_dc = Constraint(range(nbuses_dc), range(nbuses_dc), rule = voltage_diff_dc)
17
18 def cone_relaxation_dc(model, i, j):
19     return model.pij_dc[i, j]**2 <= model.lij_dc[i, j] * model.vn2_dc[i]
20 model.cone_relaxation_dc = Constraint(range(nbuses_dc), range(nbuses_dc), rule = cone_relaxation_dc)

```

In Cpp, Gurobi Cpp API does not provide built-in matrix operations. We can leverage Eigen library to enhance adding constraints with for-loops like below

```

1 // 1. Constrains for dc power flow - second-order cone relaxation
2 Eigen::MatrixXd y_dc_dense = Eigen::MatrixXd(y_dc);
3 Eigen::MatrixXd zij_dc = y_dc_dense.array().abs().cwiseInverse().matrix();
4 zij_dc -= zij_dc.diagonal().asDiagonal();
5 for (int i = 0; i < zij_dc.rows(); ++i) {
6     for (int j = 0; j < zij_dc.cols(); ++j) {
7         if (std::isinf(zij_dc(i, j))) {
8             zij_dc(i, j) = 1e4;
9         }
10    }
11 }
12
13 for (int i = 0; i < nbuses_dc; ++i) {
14     GRBLinExpr power_flow_sum = 0.0;
15     for (int j = 0; j < nbuses_dc; ++j) {
16         power_flow_sum += pij_dc(i, j);
17     }
18     model.addConstr(pn_dc(i) == power_flow_sum * pol_dc);
19 }
20
21 for (int i = 0; i < nbuses_dc; ++i) {
22     for (int j = 0; j < nbuses_dc; ++j) {
23         model.addConstr(pij_dc(i, j) + pij_dc(j, i) == zij_dc(i, j) * lij_dc(i, j));
24         model.addQConstr(pij_dc(i, j) * pij_dc(i, j) <= lij_dc(i, j) * vn2_dc(i));
25         model.addConstr(vn2_dc[i] - vn2_dc[j] == zij_dc(i, j) * (pij_dc(i, j) - pij_dc(j, i)));
26     }
27 }
28
29 for (int i = 0; i < nbuses_dc; ++i) {
30     model.addConstr(vn2_dc(i) >= 0.0);
31     for (int j = 0; j < nbuses_dc; ++j) {
32         model.addConstr(lij_dc(i, j) >= 0.0);
33     }
34 }

```

2.5.3 Establishing an objective

Establishing an optimization objective is straightforward across different programming environments. The process typically involves two key steps: first, formulating the objective function as a mathematical expression, and then incorporating it into the appropriate solver function. The code snippets regarding constructing an optimization objective in different programming environments are presented below

```

1 %% "define optimization objectives"
2 for ng = 1:ngrids
3     actgen_ac{ng} = generator_ac{ng}(:, 8);
4     if gencost_ac{ng}(1,4) == 3

```



```

5     obj_acdc{ng} = sum(actgen_ac{ng}.*( baseMVA^2*gencost_ac{ng}(:,5).*pgen_ac{ng}.^2 + baseMVA*gencost_ac{
6         ng}(:, 6).*pgen_ac{ng} + gencost_ac{ng}(:, 7) ));
7     elseif gencost_ac(1,4) == 2
8         obj_acdc{ng} = sum(actgen_ac{ng}.*( baseMVA*gencost_ac{ng}(:,5).*pgen_ac{ng} + gencost_ac{ng}(:,6) ));
9     end
10 end
11 for ng = 1:ngrids
12     Obj = Obj+obj_acdc{ng};
13 end

```

```

1  # "define optimization objectives"
2  for ng = 1:ngrids
3      actgen_ac[ng] = generator_ac[ng][:, 8]
4      if gencost_ac[ng][1, 4] == 3
5          obj[ng] = sum(actgen_ac[ng] .*
6              (baseMVA_ac^2 .* gencost_ac[ng][:, 5] .* pgen_ac[ng].^2 +
7              baseMVA_ac .* gencost_ac[ng][:, 6] .* pgen_ac[ng] +
8              gencost_ac[ng][:, 7]))
9      elseif gencost_ac[ng][1, 4] == 2
10         obj[ng] = sum(actgen_ac[ng] .*
11             (baseMVA_ac .* gencost_ac[ng][:, 5] .* pgen_ac[ng] +
12             gencost_ac[ng][:, 6]))
13     end
14 end

```

```

1  # "define optimization objectives"
2  model.objective_terms = []
3
4  for ng in range(ngrids):
5      actgen_ac = [generator_ac[ng][i, 7] for i in range(ngens_ac[ng])]
6
7      if gencost_ac[ng][0, 3] == 3:
8          obj_gencost = sum(
9              actgen_ac[i] * (
10                 baseMVA_ac**2 * gencost_ac[ng][i, 4] * model.pgen_ac[ng, i]**2 +
11                 baseMVA_ac * gencost_ac[ng][i, 5] * model.pgen_ac[ng, i] +
12                 gencost_ac[ng][i, 6]
13             ) for i in range(ngens_ac[ng]))
14      )
15
16      elif gencost_ac[ng][0, 3] == 2:
17          obj_gencost = sum(
18              actgen_ac[i] * (
19                 baseMVA_ac * gencost_ac[ng][i, 4] * model.pgen_ac[ng, i] +
20                 gencost_ac[ng][i, 5]
21             ) for i in range(ngens_ac[ng]))
22      )
23
24  model.objective_terms.append(obj_gencost)
25
26  model.objective = Objective(expr=sum(model.objective_terms), sense=minimize)

```

```

1  /* DEFINE OPTIMIZATION OBJECTIVE */
2  GRBQuadExpr obj = 0.0;
3  for (int ng = 0; ng < ngrids; ++ng) {
4      actgen_ac[ng] = generator_ac[ng].col(7);
5      if (gencost_ac[ng](0, 3) == 3) {
6          for (int i = 0; i < actgen_ac[ng].size(); ++i) {
7              obj += actgen_ac[ng](i) * (
8                  baseMVA_ac * baseMVA_ac * gencost_ac[ng](i, 4) * (pgen_ac[ng][i] * pgen_ac[ng][i]) +
9                  baseMVA_ac * gencost_ac[ng](i, 5) * pgen_ac[ng][i] +
10                  gencost_ac[ng](i, 6)
11              );
12          }
13      }
14      else if (gencost_ac[ng](0, 3) == 2) {

```

```

15     for (int i = 0; i < actgen_ac[ng].size(); ++i) {
16         obj += actgen_ac[ng][i] * (
17             baseMVA_ac * gencost_ac[ng](i, 4) * pgen_ac[ng][i] +
18             gencost_ac[ng](i, 5)
19         );
20     }
21 }
22 }
23 model.setObjective(obj);

```

2.6 Accessing the Results

By default, the results of the optimization running are printed on the screen, following the well-known Matpower display style, like below

=====							
AC Grids Bus Data							
=====							
Area	Branch	Voltage		Generation		Load	
#	#	Mag [pu]	Ang [deg]	Pg [MW]	Qg [MVar]	Pd [MW]	Qd [MVar]
----	----	-----	-----	-----	-----	-----	-----
1	1	1.060	0.000*	167.922	42.694	0.000	0.000
1	2	1.014	0.000	-	-	20.000	10.000
1	3	1.018	0.000	-	-	45.000	15.000
1	4	1.022	0.000	40.000	23.606	40.000	5.000
1	5	1.000	0.000	-	-	60.000	10.000
2	1	1.040	0.000*	76.934	103.668	0.000	0.000
2	2	1.061	0.000	121.407	163.645	0.000	0.000
2	3	0.993	0.000	-	-	0.000	0.000
2	4	1.006	0.000	-	-	0.000	0.000
2	5	1.000	0.000	84.947	-7.451	90.000	30.000
2	6	0.999	0.000	-	-	0.000	0.000
2	7	0.992	0.000	-	-	100.000	35.000
2	8	1.010	0.000	-	-	0.000	0.000
2	9	0.980	0.000	-	-	125.000	50.000
----	----	-----	-----	-----	-----	-----	-----

The total generation cost is \$9363.58/MWh(€8669.98/MWh)

=====								
AC Grids Branch Data								
=====								
Area	Branch	From	To	From Branch Flow		To Branch Flow		Branch Loss
#	#	Bus#	Bus#	Pij [MW]	Qij [MVar]	Pij [MW]	Qij [MVar]	Pij_loss [MW]
----	----	-----	-----	-----	-----	-----	-----	-----
1	1	1	2	128.979	41.985	-125.705	-32.163	3.274
1	2	1	3	38.943	6.889	-37.829	-3.549	1.114
1	3	2	3	7.839	-5.017	-7.788	5.168	0.051
1	4	2	4	8.023	-6.842	-7.958	7.036	0.065
1	5	2	5	28.531	2.761	-28.212	-1.804	0.319
1	6	3	4	0.617	-10.911	-0.606	10.946	0.012
1	7	4	5	8.564	6.367	-8.477	-6.105	0.087
2	1	1	4	76.934	103.668	-76.934	-16.397	0.000
2	2	4	5	-13.730	9.430	13.776	-9.178	0.047
2	3	5	6	15.009	-2.469	-14.918	2.862	0.090
2	4	3	6	0.000	0.000	0.000	20.156	0.000

2	5	6	7	14.918	5.259	-14.888	-5.005	0.030
2	6	7	8	-85.113	-12.389	85.752	17.804	0.639
2	7	8	2	-121.407	-7.225	121.407	163.645	0.000
2	8	8	9	35.652	12.622	-35.203	-10.364	0.449
2	9	9	4	-89.797	-16.498	90.665	23.876	0.868

The total AC network losses is 7.044 MW .

MTDC Bus Data							
Bus DC #	Bus AC #	Area #	DC Voltage Vdc [pu]	DC Power Pdc [MW]	PCC Bus Ps [MW]	Injection Qs [MVar]	Converter loss Conv_Ploss [MW]
1	2	1	1.008	60.000	61.313	40.000	1.229
2	5	1	1.000	-24.443	-23.311	1.909	1.123
3	5	2	0.998	-35.000	-33.837	-0.003	1.145

The total converter losses is 3.498 MW

MTDC Branch Data					
Branch #	From Bus#	To Bus#	From Branch Flow Pij [MW]	To Branch Flow Pij [MW]	Branch Loss Pij_loss [MW]
1	1	2	31.987	-31.725	0.262
2	2	3	7.282	-7.268	0.014
3	1	3	28.013	-27.732	0.282

The total DC network losses is 0.557 MW .

Execution time is 2.120s .

3 Mathematical Framework Implementation

3.1 Getting Start

The increasing complexity of modern power systems, especially Multi-terminal Voltage Direct Current (MT-HVDC) systems, requires an advanced simulation tool capable of addressing unique challenges posed by such systems. This project aims to implement a comprehensive mathematical framework in C++ that integrates advanced simulation techniques and aligns with border research initiatives within the power systems domain. The framework is designed to offer a wide range of functionalities, including component selection, detailed system modeling, and the setup of complex simulation scenarios, which will allow for precise and efficient stability assessments, which are critical for verifying the operational reliability of power systems incorporating both AC and DC networks. This framework will be available as an open-source package, contributing to the broader research and engineering community and ensuring accessibility for more collaboration and further advanced power system analysis research.

3.1.1 Objectives

One of the notable features of the proposed framework is performing fast and reliable stability assessments, allowing users to identify and mitigate potential system instabilities quickly. This capability is important for the operational verification of hybrid power systems like MT-HVDC-based systems, which are increasingly important in modern energy infrastructure. Besides, the framework offers detailed representations of harmonic components to increase understanding of the dynamic behavior of power systems. Minor changes in system parameters can significantly impact system stability, making this level of modeling detail crucial for accurate simulation and analysis.

Traditional Electromagnetic Transient tools (EMT), such as PSCAD, offer frequency-scanning methodologies for passive network components to analyze the steady-state harmonic amplification through the network. However, these tools are often limited in their ability to address small-signal stability issues, particularly concerning the dynamic interactions of converter controls.

The proposed framework bridges this gap by incorporating advanced simulation techniques that deliver small-signal representations of converters and their control systems. The goal is to achieve a more accurate assessment of system stability and enable a comprehensive analysis of the dynamic behavior of power systems, including harmonic interactions and stability across a wide range of operating conditions.

3.1.2 Implementation structure

The framework integrates detailed power system models of passive and active components to capture frequency-dependent behaviors across a wide frequency range. It captured a broad range of system dynamics by including detailed mathematical models of components such as transformers, impedance, and admittance, reflecting their frequency-dependent behaviors. For instance, the framework models system impedance with frequency-sensitive properties while accounting for the dynamic behavior of transformers and their effect on power flow.

The proposed framework is designed to effectively model the key components of hybrid AC and DC power systems. The architecture employs a hierarchical class structure, where the Network class manages the organization of buses and elements. The Bus and Element classes exist independently, with Element serving as the base class for a range of power system components, each implemented to reflect its unique role in the system.

The Network class represents the electrical system, managing buses and elements while storing them in hash maps for efficient access. It handles the system's topology by connecting elements to buses and organizing their relationships. The Bus class defines individual buses, which can connect to multiple elements, and the Element class represents generic electrical components capable of computing and storing their admittance matrix (Y_{matrix}) for system-level impedance and admittance calculations.

The framework employs a flexible structure for multi-phase systems, making it suitable for hybrid AC/DC grids. Additionally, it is designed to compute equivalent impedance between different parts of the system, enhancing its analytical capabilities. Key components derived from the Element class include transmission

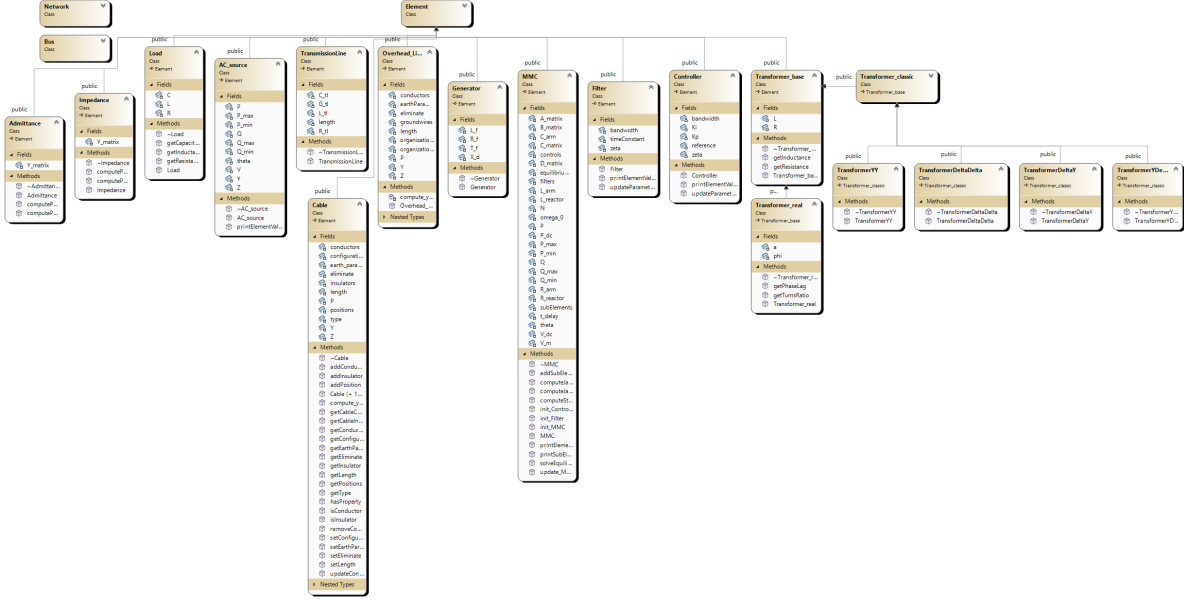


Figure 3.1: Class Diagram Overview

lines, transformers, and active elements such as power electronic devices, each modeled to capture their behavior over a wide frequency range.

The key components, derived from the Element class, include:

- **Impedances:** This class models various system components' resistive and reactive properties, such as transformers or passive filters. The impedance class allows for a precise representation of both series and parallel impedance configurations. Based on the number of pins (phases), the admittance matrix (Y_matrix) is computed for each Element, applying a uniform impedance across all phases or assigning individual impedances to each phase. This flexible approach ensures accurate modeling of frequency-dependent behavior between different network components. The impedance values can be defined numerically or symbolically, enhancing the model's capability to handle complex system dynamics.
- **Admittance:** Admittance components are included to represent the shunt elements in the system, capturing both the conductance and susceptance characteristics for accurate network modeling. These components are essential for capturing the behavior of elements such as transformers, lines, and capacitors in a network. The class supports single-phase and multi-phase systems by allowing users to define admittance values numerically or symbolically. Depending on the input, the admittance matrix is computed as either a single value applied to all phases, a vector for phase-specific admittance, or a complete matrix for multi-phase systems. This flexibility ensures an accurate and comprehensive representation of the network's static and dynamic characteristics.
- **Load:** The Load class models the demand-side characteristics of the system, representing both resistive (R), inductive (L), and capacitive (C) components in series. This flexible model captures AC and DC consumption points across multiple phases. It allows the definition of uniform or phase-specific R, L, and C values, ensuring accurate modeling of symmetrical and asymmetrical loads in power systems. The class computes the impedance and admittance matrices for the load, enabling the calculation of complex power flows and accounting for the frequency-dependent behavior of inductive and capacitive elements.
- **Generators:** Modeled as a child of the Element class, generators represent the power-producing units within both AC and DC grids.
- **Transformers:** Also derived from the Element class, transformers facilitate voltage level transformations across different system parts, essential for maintaining proper power flow and system stability.

- **Transmission Lines:** Represented by the `TransmissionLine` class, subdividing into `Cable` and `OverheadLine` classes. This distinction allows for the specific modeling of different transmission media, each with its electrical characteristics.
- **Cable:** Models underground transmission lines, capturing conductor and insulator properties, spatial configuration, and frequency-dependent impedance and admittance matrices. It supports numerical and symbolic definitions, ensuring accurate modeling of electromagnetic coupling and grounding effects.
- **Overhead Line:** Represents above-ground transmission lines, accounting for conductor resistance, inductance, and capacitance. It includes tower configurations and ground effects and supports symbolic impedance and admittance modeling for accurate power flow and stability analysis.

This structure enables detailed and scalable modeling of complex hybrid AC/DC power systems, ensuring that every component's interaction is accurately captured and simulated.

Each component has a definition for the number of input and output nodes (or connections).

3.2 Implementation and Computational Modeling of Power System Components

In power system analysis, Y-parameters (admittance parameters) are widely used to represent the relationships between currents and voltages at the terminals of multi-port network components. Y-parameters provide a convenient method for analyzing the steady-state and dynamic behavior of various power system components, including generators, loads, transmission lines, and transformers.

For each component, the Y-parameters describe how the input currents at different ports are related to the input and output voltages, making it easier to assess the impact of each component on the system's overall stability and power flow. This representation is beneficial for integrating components into larger system models, facilitating the simulation and analysis of hybrid AC and DC power systems.

This section will derive the Y-parameter representation for key power system components, starting with transmission lines.

3.2.1 Impedance

Impedance is created as a series impedance model as shown in Fig. 3.2. Impedance values can be given symbolically (dependent on angular frequency) and as a specific value.

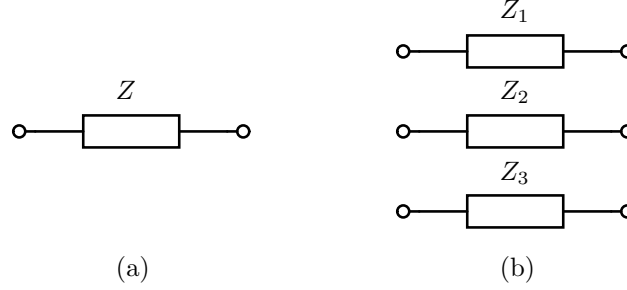


Figure 3.2: Impedance model: (a) single phase or DC; (b) three phase.

For the impedance, Y parameters are calculated as:

$$Y = \begin{bmatrix} \text{diag} \left\{ \frac{1}{Z_i} \right\} & -\text{diag} \left\{ \frac{1}{Z_i} \right\} \\ -\text{diag} \left\{ \frac{1}{Z_i} \right\} & \text{diag} \left\{ \frac{1}{Z_i} \right\} \end{bmatrix}, \quad i \in \{1, \dots, n\}, \quad (3.1)$$

where n is several phases. It must be noted here that the Y matrix is defined as a diagonal one as it is assumed that no mutual couplings exist between the three phases.

Code Explanation To further provide a detailed overview of the implementation of these impedance models in C++, we have formulated the aforesaid Y parameters using one .h and a .cpp file. The derived admittance matrix for the single-phase and the three-phase impedance model provides a clear relationship between the currents and voltages in the system.

This report provides a detailed explanation of the code implementation and structure of the `impedance.h` and the `impedance.cpp` files, which defines the `Impedance` class.

The explanation of the code is given only once now, but it follows the same structure for all components.

Explanation of ‘impedance.h’ The ‘impedance.h’ file defines the class ‘Impedance’, which is responsible for modelling the impedance and calculating the admittance matrix for a given electrical element. Below is the full content of the ‘impedance.h’ file:

```

1  #ifndef _IMPEDANCE_H_
2  #define _IMPEDANCE_H_
3
4  #include "Element.h"
5
6  /*
7   * Creates impedance with specified number of input/output pins which represent phases.
8   * The admittance expression has to be given in Omega and can have both numerical and
9   * symbolic value (example: `z = s-2`). Depending on the provided vector of impedance values,
10  * we differ two cases. Namely, we create only series impedance for each phase with values:
11  * -In the case of 1\times1 vector, impedance value is given to all diagonal impedance entries.
12  * -In the case of `pins` elements, they are representing diagonal entries of impedance.
13  */
14
15  class Impedance : public Element {
16  public:
17      /*
18       * Constructor: Impedance
19       *
20       * Constructs the impedance model with the given symbolic name, number of pins (phases),
21       * and a matrix of impedance values. It calculates the admittance matrix based on these values.

```

```

22  *
23  * Parameters:
24  * - symbol: Symbolic identifier for the impedance element (e.g., Z1, Z2)
25  * - pins: Number of input/output pins (phases)
26  * - values: DenseMatrix representing the impedance values
27  */
28  Impedance(const std::string& symbol, int pins, DenseMatrix values);
29
30  // Destructor to handle any clean-up tasks
31  ~Impedance();
32
33 private:
34  // No additional private members in this class, as the behavior is inherited from Element
35 };
36
37 #endif // _IMPEDANCE_H_

```

Listing 1: impedance.h File

Header Guards The header guards, defined by ‘`#ifndef _IMPEDANCE_H_`’, ‘`#define _IMPEDANCE_H_`’, and ‘`#endif`’, ensure that this header file is only included once during compilation, preventing any redefinition errors.

Inclusion of ‘Element.h’ The ‘`#include "Element.h"`’ statement brings in the definition of the base class ‘Element’. The ‘Impedance’ class inherits from ‘Element’, making it part of a broader system of electrical elements. This allows the ‘Impedance’ class to leverage common functionality provided by the ‘Element’ class, such as common properties for electrical components.

Class Documentation The multi-line comment above the ‘Impedance’ class provides a clear description of its purpose:

- The class models impedance for a system with a specified number of input/output pins, representing phases.
- The impedance values can be either symbolic or numerical, allowing flexibility for various use cases.
- The impedance values are assigned either uniformly (if a 1×1 vector is given) or as diagonal entries of a matrix (if multiple values are provided).

The ‘Impedance’ Class The ‘Impedance’ class inherits from the ‘Element’ class and defines the impedance behavior. The constructor takes three parameters:

- **symbol:** A string representing the symbolic name of the impedance element (e.g., Z1, Z2).
- **pins:** The number of input/output pins (phases), representing the connection points of the impedance element.
- **values:** A ‘DenseMatrix’ containing the impedance values for the different phases. This matrix is used to compute the admittance matrix.

The constructor calculates the admittance matrix based on the impedance values, which is crucial for the analysis of electrical circuits.

Destructor The destructor ‘`Impedance()`’ is included to handle any clean-up that might be necessary when an ‘Impedance’ object is destroyed. Although no specific resources are allocated in this class, the destructor is good practice in case the implementation expands in the future.

Private Members There are no additional private members defined in this class. The necessary properties and methods are inherited from the ‘Element’ class.

Introduction This report further provides a detailed explanation of the ‘impedance.cpp’ file, which contains the implementation of the ‘Impedance’ class defined in ‘impedance.h’. The implementation includes the constructor that calculates the admittance matrix (‘Y_matrix’) for a given impedance and the destructor that handles clean-up when the object is destroyed. The ‘Impedance’ class inherits from the ‘Element’ class and models multi-phase impedance elements.

Explanation of ‘impedance.cpp’ The ‘impedance.cpp’ file implements the functionality for initializing an impedance object and calculating the corresponding admittance matrix. The code also handles different cases based on the number of impedance values provided and ensures that the correct admittance matrix is constructed for single-phase or multi-phase systems.

```

1  #include "Impedance.h"
2
3  /*
4   * Impedance Constructor
5   *
6   * Initializes the impedance element by assigning the admittance matrix (Y_matrix) based on
7   * the number of pins and impedance values provided. Depending on whether one value or multiple
8   * values are provided, the constructor creates either identical or phase-specific impedance entries.
9   */
10 Impedance::Impedance(const std::string& symbol, int pins, DenseMatrix values):
11     Element(symbol, pins, pins) {
12     // Check if impedance values are provided
13     if (values.ncols() != 0) // if there are entries
14     {
15         if (pins > 0) { // Check for valid number of pins
16             // Case 1: Single impedance value provided for all phases
17             if (values.ncols() == 1) {
18                 for (int i = 0; i < pins; i++)
19                     Y_matrix.set(i, i, div(integer(1), values.get(0, 0)));
20             }
21             // Case 2: Multiple values provided for each phase's impedance
22             else if (values.ncols() == pins) {
23                 for (int i = 0; i < pins; i++)
24                     Y_matrix.set(i, i, div(integer(1), values.get(0, i)));
25             }
26             // Error: The number of impedance values provided doesn't match the number of phases
27             else
28                 throw invalid_argument("Invalid number of admittance vector entries: " + values.ncols());
29         }
30     }
31     else
32         throw invalid_argument("Invalid number of pins, must be greater than 0!");
33
34     // Fill in the complete Y parameters
35     for (int i = 0; i < pins; i++)
36         for (int j = 0; j < pins; j++) {
37             Y_matrix.set(pins + i, j, sub(zero, Y_matrix.get(i, j)));
38             Y_matrix.set(pins + i, pins + j, Y_matrix.get(i, j));
39             Y_matrix.set(i, pins + j, sub(zero, Y_matrix.get(i, j)));
40         }
41 }
42
43 // Destructor
44 Impedance::~Impedance() {
45     // Clean-up if needed (Smart pointers handle most memory management)
46 }
47

```

Listing 2: impedance.cpp File

Constructor: ‘Impedance’ The ‘Impedance’ constructor is responsible for initializing the impedance element and calculating the corresponding admittance matrix (‘Y_matrix’). The constructor accepts the following parameters:

- **symbol:** A string representing the symbolic name of the impedance element (e.g., Z1, Z2).
- **pins:** An integer representing the number of phases (or input/output pins) of the impedance element.
- **values:** A ‘DenseMatrix’ object representing the impedance values for each phase. Depending on the number of impedance values provided, the constructor handles two cases.

The constructor first checks if any impedance values are provided by checking the number of columns in the ‘values’ matrix. Then, it checks if the number of pins is greater than 0, ensuring that the impedance element has a valid number of phases.

Case 1: Single Impedance Value (single phase cases) If only one impedance value is provided, the constructor assumes that this value applies to all phases equally. The admittance matrix (‘Y_matrix’) is calculated as the reciprocal of the impedance value for each diagonal entry, as seen in the following loop:

```
1 for (int i = 0; i < pins; i++)
2   Y_matrix.set(i, i, div(integer(1), values.get(0, 0)));
```

Case 2: Multiple Impedance Values (multi-phase or three-phase cases) If multiple impedance values are provided, each phase gets its own specific impedance value. The constructor iterates through each phase and assigns the reciprocal of the respective impedance value to the diagonal entries of the admittance matrix:

```
1 for (int i = 0; i < pins; i++)
2   Y_matrix.set(i, i, div(integer(1), values.get(0, i)));
```

Error Handling If the number of impedance values provided doesn’t match the number of phases, an exception is thrown to handle the error:

```
1 throw invalid_argument("Invalid number of admittance vector entries: " + values.ncols());
```

Filling the Complete Admittance Matrix After setting the diagonal entries, the constructor proceeds to fill the complete admittance matrix (‘Y_matrix’). It ensures symmetry between the input and output phases by setting the corresponding off-diagonal elements. The negative values ensure proper alignment of phase admittances:

```
1 for (int i = 0; i < pins; i++)
2   for (int j = 0; j < pins; j++) {
3     Y_matrix.set(pins + i, j, sub(zero, Y_matrix.get(i, j)));
4     Y_matrix.set(pins + i, pins + j, Y_matrix.get(i, j));
5     Y_matrix.set(i, pins + j, sub(zero, Y_matrix.get(i, j)));
6   }
```

Destructor: ‘Impedance’ The destructor is responsible for clean-up tasks when an ‘Impedance’ object is destroyed. In this implementation, memory management is primarily handled by smart pointers, so no explicit resource release is required.

Power Flow Implementation The power flow computation is essential for analyzing electrical networks. In this section, we describe the implementation of power flow calculations for AC and DC networks within the Impedance class.

AC Power Flow Calculation

The function `computePowerFlowAC` computes the power flow parameters for AC networks. It extracts the required branch parameters and evaluates the equivalent impedance based on the admittance matrix.

```

1 void Impedance::computePowerFlowAC(std::map<std::string, std::map<std::string, double>>& branchData,
2 std::map<std::string, double>& globalParams) const {
3     int key = branchData.size(); // Unique branch identifier
4     branchData[std::to_string(key)]["transformer"] = 0;
5     branchData[std::to_string(key)]["tap"] = 1.0;
6     branchData[std::to_string(key)]["shift"] = 0.0;
7     branchData[std::to_string(key)]["c_rating_a"] = 1.0;
8
9     // Compute Y parameters at operational frequency
10    std::complex<double> s = globalParams["omega"] * std::complex<double>(0, 1);
11
12    // Convert SymEngine expression to double
13    double Y_00_real = SymEngine::eval_double(*Y_matrix.get(0, 0));
14    std::complex<double> Y_00(Y_00_real, 0.0);
15
16    if (Y_00 == std::complex<double>(0, 0)) {
17        throw std::runtime_error("Y_matrix(0,0) is zero, division by zero error.");
18    }
19
20    std::complex<double> Z_eq = std::complex<double>(1.0) / Y_00 / globalParams["Z_base"];
21
22    branchData[std::to_string(key)]["br_r"] = std::real(Z_eq);
23    branchData[std::to_string(key)]["br_x"] = std::imag(Z_eq);
24    branchData[std::to_string(key)]["g_fr"] = 0;
25    branchData[std::to_string(key)]["b_fr"] = 0;
26    branchData[std::to_string(key)]["g_to"] = 0;
27    branchData[std::to_string(key)]["b_to"] = 0;
28 }

```

Listing 3: AC Power Flow Computation

DC Power Flow Calculation

Similarly, the function `computePowerFlowDC` computes the power flow parameters for DC networks. It evaluates the equivalent resistance by computing the impedance at a very low frequency.

```

1 void Impedance::computePowerFlowDC(std::map<std::string, std::map<std::string, double>>& branchDCData,
2 std::map<std::string, double>& globalParams) const {
3     int key = branchDCData.size(); // Unique DC branch identifier
4     branchDCData[std::to_string(key)]["l"] = 0.0;
5     branchDCData[std::to_string(key)]["c"] = 0.0;
6
7     // Compute Y parameters at low frequency (DC)
8     std::complex<double> s = std::complex<double>(0, 1e-6);
9
10    // Convert SymEngine expression to double
11    double Y_00_real = SymEngine::eval_double(*Y_matrix.get(0, 0));
12    std::complex<double> Y_00(Y_00_real, 0.0);
13
14    if (Y_00 == std::complex<double>(0, 0)) {
15        throw std::runtime_error("Y_matrix(0,0) is zero, division by zero error.");
16    }
17
18    std::complex<double> Z_eq = std::complex<double>(1.0) / Y_00 / globalParams["Z_base"];
19
20    branchDCData[std::to_string(key)]["r"] = std::real(Z_eq);
21 }

```

Listing 4: DC Power Flow Computation

The above implementations ensure that the impedance model is properly integrated into power flow computations for both AC and DC networks, allowing for accurate analysis in a multi-phase simulation framework.

3.2.2 Admittance

An admittance is constructed as an element by calling the constructor: `Admittance(const std::string& symbol, int pins, DenseMatrix values)`.

This constructor creates an admittance element with a specified number of input/output pins. The number of input pins is equal to the number of output pins. The values provided for admittance can either be numerical or symbolic, and they are passed as a matrix. The pins are labeled as 1.1, 1.2, ..., 1.n for input pins, and 2.1, 2.2, ..., 2.n for output pins, where **n** is the number of pins. The admittance values are given in Siemens and can have symbolic values (e.g., `values = 1/s` for a symbolic admittance where **s** is the Laplace variable).

For the case of a single-phase system (1x1 impedance), the values matrix will contain a single entry.

Example: `Admittance("Z", 1, DenseMatrix{1000})` creates a single-phase admittance with a value of 1000 Siemens.

If the admittance is multiport (i.e., the number of input/output pins is greater than 1), the values matrix can either be a scalar, vector, or full matrix:

- If the values matrix has a single entry, the admittance is a diagonal matrix with equal values on the diagonal.
- If the values matrix has a length equal to the number of pins, only the diagonal entries of the admittance matrix will be nonzero, with values corresponding to the array.
- If the values matrix is of size $n \times n$, the admittance matrix will be fully populated according to the array, representing phase interactions.

For example, a multi-phase system can be represented as:

`Admittance("Z", 3, DenseMatrix{[1000, 2000, 3000]})` creates a three-phase admittance with different diagonal values for each phase.

In this example, the Laplace operator **s** can be used to describe the frequency-dependent behavior of the system. The admittance may also be represented as a transfer function in terms of **s**, which can be either polynomial, nonlinear or include non-integer powers of **s**. For example, a time delay might be represented as an exponential function of **s**.

The code below can be used to define the network with its impedances, as depicted in Fig. 3.3, and to generate its bode plot.

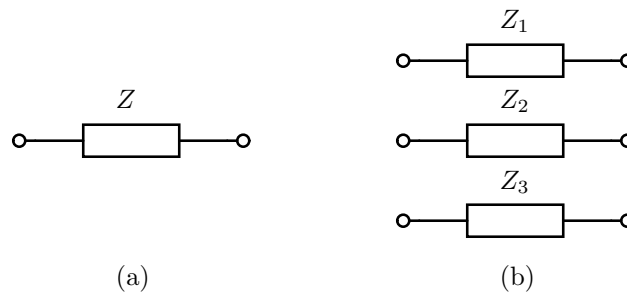


Figure 3.3: Impedance model: (a) single phase or DC; (b) three phase.

This document explains the `admittance.h` header file. The `Admittance` class represents an electrical component's admittance in a circuit. It inherits from the base class `Element` and constructs the admittance matrix for a system based on input parameters such as symbolic names, the number of pins (phases), and admittance values. Admittance values can be either numerical or symbolic.

Code Explanation The `Admittance` class is defined as a derived class from the `Element` class. It handles the creation of the admittance matrix for a given number of input/output pins (phases) based on symbolic or numerical values.

Constructor The constructor for the `Admittance` class initializes an admittance element based on the provided symbolic name, number of pins (phases), and the admittance values. The values could represent different configurations based on the number of pins (single-phase or multi-phase).

The `Admittance` constructor initializes the object by populating the `Y_matrix` based on the provided number of pins and admittance values. It handles three different configurations: a single admittance value, a vector of admittance values for each phase, or a full admittance matrix for multi-phase systems. The class ensures that errors are properly handled when inputs are invalid, and the destructor takes care of any cleanup tasks when the object is destroyed.

First, the constructor checks if any admittance values have been provided. If there are no values or the number of pins is not greater than 0, an exception is thrown.

- **Case 1: Single Admittance Value (single-phase system):** If a single admittance value is provided, it is applied to all diagonal entries in the `Y_matrix`.
- **Case 2: Vector of Admittance Values (multi-phase or three-phase cases):** If a vector of admittance values is provided, it populates the diagonal of the `Y_matrix`, with each entry corresponding to a specific phase.
- **Case 3: Full Admittance Matrix:** If a full admittance matrix is provided, each entry is used to populate the corresponding position in the `Y_matrix`. This is suitable for complex multi-phase systems.
- **Error Handling:** If the provided admittance values do not match any expected configuration (single value, vector, or full matrix), an exception is thrown, indicating an invalid number of values.

After processing the admittance values, the constructor fills the complete `Y_matrix` by adjusting the off-diagonal values. This is done to ensure the matrix properly represents the relationships between different phases.

Destructor The destructor ensures that any memory allocated or resources used by the `Admittance` object are properly released when the object is destroyed. The destructor is declared but not defined within the header file, as the behavior is inherited from the `Element` class, which likely handles memory management. In this case, manual memory management is not required since the `DenseMatrix` and other standard library components automatically manage memory.

Parameters

- **symbol:** A symbolic identifier for the admittance element. This is used to distinguish between different admittance components (e.g., `Y1`, `Y2`).
- **pins:** The number of pins (or phases) that the admittance element connects to. This number defines the size of the matrix representing the admittance.
- **values:** A `DenseMatrix` object that holds the actual admittance values. This matrix could represent a single-phase or multi-phase configuration depending on its size.

Private Members No additional private members are defined in the `Admittance` class. The class relies on the functionality inherited from the `Element` base class.

Power Flow Implementation The power flow computation is essential for analyzing electrical networks. In this section, we describe the implementation of power flow calculations for AC and DC networks within the `Admittance` class.

AC Power Flow Calculation

For AC networks, we compute the power flow using the admittance matrix. The following C++ function computes the power flow for an AC network. It takes in the branch data and global parameters, and updates the branch data with relevant power flow parameters. The function calculates the equivalent impedance Z_{eq} based on the admittance matrix and updates the branch parameters.

```

1 void Admittance::computePowerFlowAC(std::map<std::string, std::map<std::string, double>>& branchData,
2 std::map<std::string, double>& globalParams) const {
3     int key = branchData.size(); // Unique branch identifier
4     branchData[std::to_string(key)]["transformer"] = 0;
5     branchData[std::to_string(key)]["tap"] = 1.0;
6     branchData[std::to_string(key)]["shift"] = 0.0;
7     branchData[std::to_string(key)]["c_rating_a"] = 1.0;
8
9     // Convert SymEngine expression to double
10    double Y_00_real = SymEngine::eval_double(*Y_matrix.get(0, 0));
11    std::complex<double> Y_00(Y_00_real, 0.0);
12
13    if (Y_00 == std::complex<double>(0, 0)) {
14        throw std::runtime_error("Y_matrix(0,0) is zero, division by zero error.");
15    }
16
17    std::complex<double> Z_eq = std::complex<double>(1.0) / Y_00 / globalParams["Z_base"];
18
19    branchData[std::to_string(key)]["br_r"] = std::real(Z_eq);
20    branchData[std::to_string(key)]["br_x"] = std::imag(Z_eq);
21    branchData[std::to_string(key)]["g_fr"] = std::real(Y_00);
22    branchData[std::to_string(key)]["b_fr"] = std::imag(Y_00);
23    branchData[std::to_string(key)]["g_to"] = std::real(Y_00);
24    branchData[std::to_string(key)]["b_to"] = std::imag(Y_00);
25 }

```

Listing 5: AC Power Flow Computation

DC Power Flow Calculation

For DC networks, the power flow computation is simplified to account for the resistance of the branches. The following function computes the power flow for DC networks and updates the branch DC data with the equivalent resistance.

```

1 void Admittance::computePowerFlowDC(std::map<std::string, std::map<std::string, double>>& branchDCData,
2 std::map<std::string, double>& globalParams) const {
3     int key = branchDCData.size(); // Unique DC branch identifier
4     branchDCData[std::to_string(key)]["l"] = 0.0;
5     branchDCData[std::to_string(key)]["c"] = 0.0;
6
7     // Convert SymEngine expression to double
8     double Y_00_real = SymEngine::eval_double(*Y_matrix.get(0, 0));
9     std::complex<double> Y_00(Y_00_real, 0.0);
10
11    if (Y_00 == std::complex<double>(0, 0)) {
12        throw std::runtime_error("Y_matrix(0,0) is zero, division by zero error.");
13    }
14
15    std::complex<double> Z_eq = std::complex<double>(1.0) / Y_00 / globalParams["Z_base"];
16
17    branchDCData[std::to_string(key)]["r"] = std::real(Z_eq);
18 }

```

Listing 6: DC Power Flow Computation

The core of the power flow computation lies in the admittance matrix, which models the relationship between currents and voltages in the network. In the AC power flow computation, the admittance matrix is used to calculate the equivalent impedance Z_{eq} , which is then used to update the branch data with parameters such as resistance (r), reactance (x), and conductance (g). In DC networks, only the resistance is considered, as there is no reactive component in DC power flow.

The computation of the power flow for both AC and DC networks is essential for simulating and analyzing the behavior of power systems, especially in the context of multi-phase systems where the network's configuration may vary.

3.2.3 AC Source

Introduction An AC source can be characterized by its series impedance between terminals. The impedance between each terminal and the source is defined in the impedance matrix $\mathbf{Z}$, and the admittance matrix $\mathbf{Y}$ is derived from $\mathbf{Z}$ by inverting the impedance values.

Mathematical Model Let Z_{ii} be the impedance at terminal i . The self-admittance at terminal i is defined as $Y_{ii} = \frac{1}{Z_{ii}}$. This represents the relationship between the voltage and current at terminal i . For an AC source with n terminals (pins), we can generalize this for each terminal.

If the impedance is uniform for all terminals, i.e., $Z_{ii} = Z$ for all i , the self-admittance becomes $Y_{ii} = \frac{1}{Z}$ for all i .

Y Parameters For an AC source with n terminals, we first construct the diagonal elements of the $\mathbf{Y}$ matrix using the series impedance. This means setting:

$$Y_{ii} = \frac{1}{Z}$$

for each terminal i . The matrix starts with the form:

$$\mathbf{Y} = \begin{bmatrix} Y_{11} & 0 & 0 & \cdots & 0 \\ 0 & Y_{22} & 0 & \cdots & 0 \\ 0 & 0 & Y_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & Y_{nn} \end{bmatrix}$$

where $Y_{ii} = \frac{1}{Z_{ii}}$.

To represent the interaction between the terminals, we must introduce off-diagonal terms. These terms are generally set to zero initially, as the mutual impedance is typically not considered for a simple AC source with independent terminals. However, in more complex scenarios, the mutual admittance could be included.

Additional rows and columns are added to account for the terminals' connections to the source to complete the $\mathbf{Y}$ matrix. The generalized $\mathbf{Y}$ matrix is formulated as:

$$\mathbf{Y} = \begin{bmatrix} Y_{11} & 0 & 0 & \cdots & 0 & -Y_{11} & 0 & \cdots & 0 \\ 0 & Y_{22} & 0 & \cdots & 0 & 0 & -Y_{22} & \cdots & 0 \\ 0 & 0 & Y_{33} & \cdots & 0 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & Y_{nn} & 0 & 0 & \cdots & -Y_{nn} \\ -Y_{11} & 0 & 0 & \cdots & 0 & Y_{11} & 0 & \cdots & 0 \\ 0 & -Y_{22} & 0 & \cdots & 0 & 0 & Y_{22} & \cdots & 0 \\ 0 & 0 & 0 & \cdots & -Y_{nn} & 0 & 0 & \cdots & Y_{nn} \end{bmatrix}$$

The off-diagonal negative terms (e.g., $-Y_{ii}$) are introduced to account for the interaction between the terminals and ensure that the system satisfies Kirchhoff's Current Law (KCL), which states that the sum of currents entering and leaving a node must equal zero.

Code Explanation The code for the 'AC_source.h' provides a general description of the AC source model. The AC source is modeled as an ideal voltage source with a series impedance. The source can be single-phase or multi-phase, and the number of terminals (pins) is passed as an input. The power flow parameters, such as voltage magnitude, phase shift, and power output limits, can be added after the source is created. The 'AC_source' class inherits from the 'Element' class.

Private Properties The private properties are as follows:

- **Z**: A ‘DenseMatrix’ object that stores the series impedance of the source, in ohms. This allows for modeling impedance in multi-phase systems.
- **V**: Represents the voltage magnitude in kilovolts (kV).
- **theta**: Represents the phase angle in radians.
- **P**: The active power output of the source in megawatts (MW).
- **Q**: The reactive power output of the source in megavolt-amperes reactive (MVar).
- **P_min, P_max**: The minimum and maximum allowable active power output in megawatts.
- **Q_min, Q_max**: The minimum and maximum allowable reactive power output in MVar.

These power flow properties are essential for power flow analysis in electrical networks. They help define operational limits for the AC source when performing simulations.

Constructor Takes three parameters: - ‘symbol’: A string representing the symbol of the AC source. - ‘pins’: The number of terminals (or phases) of the AC source. - ‘Z’: A ‘DenseMatrix’ representing the series impedance matrix of the source. This allows for defining a more complex impedance between the terminals. The constructor initializes the ‘AC_source’ object with the given parameters.

Destructor The destructor ‘AC_source()’ is defined but doesn’t perform any special operations, as most memory management is handled automatically.

3.2.4 Load

Introduction This report provides a detailed description of a load model undertaken in this project consisting of resistive (R), inductive (L), and capacitive (C) components in series for both single-phase and three-phase systems. The objective is to model the admittance matrix Y for a three-phase system, which characterizes the relationship between currents and voltages in the system.

Series RLC Circuit for Single-Phase We will first start with the single-phase model, derive the Y parameters, and extend to three-phase models. In a single-phase series RLC circuit, the total impedance Z is given by the sum of the resistive, inductive, and capacitive impedances:

$$Z = R + j\omega L - \frac{j}{\omega C}$$

The admittance Y is the inverse of the impedance:

$$Y = \frac{1}{Z} = \frac{1}{R + j\omega L - \frac{j}{\omega C}}$$

The corresponding circuit for a single-phase series RLC load is shown in Figure 3.4.

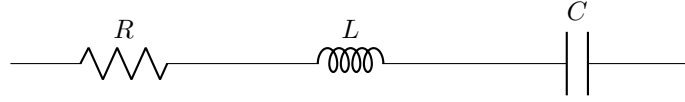


Figure 3.4: Single-Phase Series RLC Load

Three-Phase Series RLC Circuit Model For a balanced three-phase system, we assume that each phase has its own resistive, inductive, and capacitive components, and these components are connected in series. For the three-phase case, it is assumed that there exists no mutual couplings between the loads of the three phases.

For phase a , phase b , and phase c , the impedance for each phase is defined as:

$$Z_a = R_a + j\omega L_a - \frac{j}{\omega C_a}$$

$$Z_b = R_b + j\omega L_b - \frac{j}{\omega C_b}$$

$$Z_c = R_c + j\omega L_c - \frac{j}{\omega C_c}$$

Y Parameters The admittance matrix Y for a three-phase system is a 3×3 matrix that relates the current vector $I = [I_a, I_b, I_c]^T$ and the voltage vector $V = [V_a, V_b, V_c]^T$ as follows:

$$\begin{bmatrix} I_a \\ I_b \\ I_c \end{bmatrix} = \begin{bmatrix} Y_{aa} & Y_{ab} & Y_{ac} \\ Y_{ba} & Y_{bb} & Y_{bc} \\ Y_{ca} & Y_{cb} & Y_{cc} \end{bmatrix} \begin{bmatrix} V_a \\ V_b \\ V_c \end{bmatrix}$$

For an uncoupled three-phase system, the off-diagonal elements Y_{ab}, Y_{ac}, Y_{ba} , etc., are zero, and the diagonal elements are the admittance of the respective phases.

Balanced Three-Phase System For a balanced three-phase system, we assume that the impedance of each phase is the same:

$$R_a = R_b = R_c = R, \quad L_a = L_b = L_c = L, \quad C_a = C_b = C_c = C$$

The diagonal elements of the admittance matrix are:

$$Y_{aa} = Y_{bb} = Y_{cc} = \frac{1}{R + j\omega L - \frac{j}{\omega C}}$$

This simplifies the admittance matrix for a balanced three-phase system to:

$$Y = \begin{bmatrix} Y_{aa} & 0 & 0 \\ 0 & Y_{aa} & 0 \\ 0 & 0 & Y_{aa} \end{bmatrix} = \begin{bmatrix} \frac{1}{R+j\omega L-\frac{j}{\omega C}} & 0 & 0 \\ 0 & \frac{1}{R+j\omega L-\frac{j}{\omega C}} & 0 \\ 0 & 0 & \frac{1}{R+j\omega L-\frac{j}{\omega C}} \end{bmatrix}$$

It must be noted here that for most of the cases, this Y matrix for the three-phase load is diagonal as the mutual coupling between phases is negligible.

Code Explanation This report provides a detailed explanation of the code implementation and structure of the `load.h` and the `load.cpp` files, which defines the `Load` class. The class models a three-phase series RLC (Resistor-Inductor-Capacitor) load, allowing the user to specify resistance, inductance, and capacitance values either for each phase separately or for a balanced load scenario.

The `Load` class represents an electrical load with resistive, inductive, and capacitive components connected in series. It is designed to handle multiple phases, where each phase can have distinct values for resistance (R), inductance (L), and capacitance (C). The `Load` object is derived from the `Element` base class and offers methods to retrieve the RLC values for each phase.

Private Data Members The ‘Load’ class inherits from the ‘Element’ class, and its primary function is to define the parameters of an electrical load, including resistance (R), inductance (L), and capacitance (C) for multiple phases.

The class maintains three private `std::vector<double>` members to store the values of resistance (R), inductance (L), and capacitance (C) for each phase.

Constructor The constructor of the ‘Load’ class initializes the load parameters based on the input values. The constructor checks the validity of the input parameters and initializes the R, L, and C values based on the number of pins specified. The constructor throws exceptions if the number of pins is invalid or if the size of the values vector does not match the expected capacity.

After initializing the parameters, the constructor validates them to ensure they are non-negative and properly initialized. This part of the code ensures that any load with invalid parameters will raise an error message indicating which phase is problematic.

The next part of the constructor calculates the Y parameters for each phase based on the initialized R, L, and C values, which is stored in the ‘*Y_matrix*’.

Getter Methods The getter methods allow users to retrieve the resistance, inductance, and capacitance values for a specific phase. These methods check if the requested phase index is valid. If the index is invalid, an exception is thrown.

3.2.5 Transformer with magnetization resistance and inductance

Introduction A transformer is an electrical device that transfers electrical energy between two or more circuits through electromagnetic induction. These transformers are commonly employed in power distribution networks, residential and commercial buildings, and various industrial applications, such as supplying power to lighting systems, heating equipment, and small motors. The possible ratings of single-phase transformers typically range from a few VA (volt-amperes) for small transformers used in electronic applications to several kVA (kilovolt-amperes) for larger transformers in distribution systems.

This model can be used to model the single-phase and three-phase YY transformers as given in [3].

Mathematical Modeling of a Transformer A single-phase transformer consists of a primary winding and a secondary winding. The voltage and current relationships can be expressed as:

$$\frac{V_p}{V_s} = \frac{N_p}{N_s} = a \exp(j\phi),$$

$$-\frac{I_p}{I_s} = \frac{N_p}{N_s} = \frac{1}{a} \exp(-j\phi),$$

where the values are:

- V_P, I_P - primary voltage and current;
- V_S, I_S - secondary voltage and current;
- N_P, N_S - number of turns in the primary and secondary winding;
- $a = \frac{N_P}{N_S}$ - turns ratio;
- ϕ - phase shift;
- L_P and R_P are inductance and resistance of the primary winding;
- L_S and R_S are inductance and resistance of the secondary winding.

The transformer model is depicted in Fig. 3.5. Furthermore, $Z_P = R_P + j\omega L_P$ and $Z_S = R_S + j\omega L_S$ are the

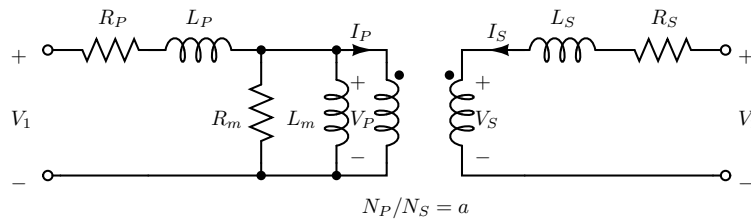


Figure 3.5: Single-phase transformer with neglected phase shift.

equivalent impedances seen from the primary and secondary sides, and $Y_m = \frac{1}{R_m} + \frac{1}{j\omega L_m}$ is the equivalent magnetization admittance [4].

Y Parameters For the transformer depicted in Fig. 3.5, the Y-parameters are as follows:

$$Y = \begin{bmatrix} \frac{1}{Z_P + \frac{1}{Y_m + \frac{1}{(a \exp(j\phi))^2 Z_S}}} & \frac{-(a \exp(j\phi))}{Z_P + (a \exp(j\phi))^2 Z_S (Y_m Z_P + 1)} \\ \frac{-(a \exp(j\phi))}{Z_P + (a \exp(j\phi))^2 Z_S (Y_m Z_P + 1)} & \frac{1}{Z_S + \frac{1}{(a \exp(j\phi))^2 (Y_m + \frac{1}{Z_P})}} \end{bmatrix}. \quad (3.2)$$

Code Explanation This report provides a detailed explanation of the **Transformer** class implemented in C++. The **Transformer** class is a subclass of **Element** and is used to model the behavior of transformers in power systems.

Public Methods

- `Transformer(const std::string& symbol, int pins, const std::vector<double>& values)`: Constructor that initializes the transformer with a symbol, the number of pins, and values.

Constructor initializes the transformer with a given symbol, number of pins, and values. Furthermore, it validates that none of the parameters are zero and represents that all the parameters are defined and initialized correctly.

The constructor defines symbolic resistances and reactances and computes the admittance for the primary and secondary sides. The Y-parameters are then computed and stored in a matrix *Y_matrix*. It must be noted here that for this formulation, it is assumed that there is no mutual coupling existing between the phases (as in the code the number of pins can be modeled as the number of phases this model is intended to work for), hence this *Y_matrix* for the multiphase system can be defined as a diagonal one. This represents a simplistic case for analysis and this model is more suitable for a single phase transformer case.

- `~Transformer()` **override**: Destructor that overrides the base class destructor.

Destructor outputs a message when a transformer object is destroyed.

- `double getResistance(int winding) const`: Returns the resistance of the specified winding. It checks for valid winding indices and throws an exception if invalid.
- `double getInductance(int winding) const`: Returns the reactance of the specified winding. Similar to `getResistance`, it checks for valid indices.
- `double getTurnsRatio() const`: Returns the turns ratio of the transformer.
- `double getPhaseLag()`: Returns the phase lag of the transformer.

Private Member Variables

- `std::vector<double> R`: A vector that stores the resistances for both primary and secondary windings, and for magnetization resistance if given.
- `std::vector<double> L`: A vector that stores the inductances for both primary and secondary windings, and for magnetization inductance if given.
- `double a`: A variable that represents the turn ratio of the transformer.
- `phi`: The phase lag in radians.

3.2.6 Transformers with mutual inductance

This transformer model which includes mutual inductance is depicted for a single phase in Fig. 3.6. It is also considered a classic transformer model. Taking into account modeling from [5, 3], the following parameters were used:

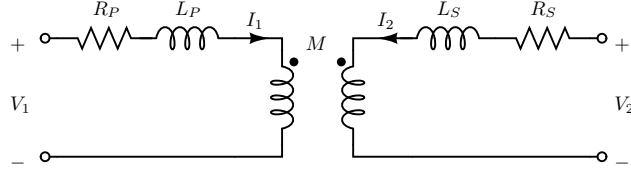


Figure 3.6: Classic transformer model.

- $Z_P = R_P + j\omega L_P$ are the primary winding impedances;
- $Z_S = R_S + j\omega L_S$ are the secondary winding impedances;
- M are the mutual inductances.

In this formulation, the admittance matrix accounts for the coupling between the primary and secondary windings, as well as the phase shifts introduced by the transformer configuration. The matrix provides a comprehensive representation of the transformer's electrical characteristics in terms of its admittances and mutual reactances.

In this case, it is easy to derive Z parameters as:

$$Z = \begin{bmatrix} Z_P & j\omega M \\ j\omega M & Z_S \end{bmatrix}. \quad (3.3)$$

Then, Y parameters can be estimated as an inverse to give:

$$Y = \begin{bmatrix} \frac{Z_S}{Z_P Z_S - (j\omega M)^2} & -\frac{j\omega M}{Z_P Z_S - (j\omega M)^2} \\ -\frac{j\omega M}{Z_P Z_S - (j\omega M)^2} & \frac{Z_P}{Z_P Z_S - (j\omega M)^2} \end{bmatrix}, \quad (3.4)$$

as estimated in [6] page 317.

Class Definition: ‘Transformer_real’ is a derived class from the ‘Transformer_base’ base class. It represents a single-phase, three-phase, or any multiple-phase transformer with key electrical characteristics, including winding resistances, inductances, turns ratio, and phase lag.

Constructor: The constructor takes the following parameters:

- **symbol:** A string representing the transformer’s name or symbol.
- **pins:** The number of electrical pins in the transformer.
- **values:** A vector of doubles representing resistance, reactance, and mutual inductance.

Destructor: The destructor is overridden from the base class to manage the cleanup of any resources used by this class, though no special resource handling is required here.

Accessor Methods: Several methods provide access to key transformer properties:

- **getMutualInductance():** Retrieves the mutual inductance.

Private Members:

- **M:** A double representing the mutual inductance between windings.

YY transformer

Introduction The YY transformer is a vital component in power systems, primarily used for voltage level conversion between the star (Y) configuration. The star sides are typically grounded, providing system stability and facilitating the management of unbalanced loads. This transformer configuration is particularly beneficial for applications requiring voltage transformation, improved system reliability, and effective load balancing.

This report focuses on the mathematical model of a generalized YY transformer, incorporating the phase shift factors and the mutual and cross-couplings between the phases.

Applications The YY configuration finds applications in various domains, including:

- **Power Distribution:** Transforming voltages for efficient distribution to various loads.
- **Load Balancing:** Managing phase imbalances in power systems to enhance overall system performance.
- **Harmonic Mitigation:** Reducing harmonics in the electrical network through proper winding connections.

Mathematical Modeling The Y parameters (admittance matrix) are crucial for defining the relationships between phase currents and voltages at the transformer terminals. A comprehensive model includes mutual and cross-phase couplings, alongside phase shift factors.

The primary side phase voltages for the star configuration are:

$$V_{pa} = V_{aN1}, \quad V_{pb} = V_{bN1}, \quad V_{pc} = V_{cN1}, \quad (3.5)$$

where V_{aN}, V_{bN}, V_{cN} represent the balanced three-phase voltages with respect to ground. The secondary side (star) voltages also correspond to the phase voltages:

$$V_{sa} = V_{aN2}, \quad V_{sb} = V_{bN2}, \quad V_{sc} = V_{cN2}. \quad (3.6)$$

Furthermore, the currents on both primary and secondary sides are the same as for the primary and secondary side transformer currents. Namely, it values that:

$$I_{pa} = I_{a1}, \quad I_{pb} = I_{b1}, \quad I_{pc} = I_{c1} \quad (3.7a)$$

$$I_{sa} = I_{a2}, \quad I_{sb} = I_{b2}, \quad I_{sc} = I_{c2}. \quad (3.7b)$$

Depending on the transformer design and the model considered, to construction. YY transformer can be used as a classic or real transformer, with just given information that it is

This model provides a detailed representation of the Star-Star transformer, capturing the essential interactions and relationships between the phases while accounting for phase shifts and mutual coupling effects.

Y Matrix The Y matrix for a three-phase YY transformer is the same as for the single-phase transformer with mutual inductance, just with 6×6 size as given here:

$$Y = \begin{bmatrix} \text{diag} \left\{ \frac{Z_S}{Z_P Z_S - (j\omega M)^2} \right\}_{3 \times 3} & -\text{diag} \left\{ \frac{j\omega M}{Z_P Z_S - (j\omega M)^2} \right\}_{3 \times 3} \\ -\text{diag} \left\{ \frac{j\omega M}{Z_P Z_S - (j\omega M)^2} \right\}_{3 \times 3} & \text{diag} \left\{ \frac{Z_P}{Z_P Z_S - (j\omega M)^2} \right\}_{3 \times 3} \end{bmatrix}. \quad (3.8)$$

Code Explanation The `TransformerYY` class inherits from the `Transformer_classic` class and includes several methods to represent and interact with the transformer characteristics. C++ code implemented for modeling a three-phase Y–Y transformer is available in files `transformer_Y_Y.cpp` and `transformer_Y_Y.h`

Constructor The constructor initializes the transformer parameters using the provided values, mutual inductances, and coupling coefficients. It checks for valid sizes for the vectors and throws exceptions if the conditions are not met.

Destructor The destructor cleans up resources when an object of the **TransformerYY** class is destroyed.

Member Variables The private member variables are the same as for the parent class.

Y – Δ transformer

Introduction The Y – Δ transformer is a fundamental component in power systems, serving to convert voltage levels between the star (Y) configuration and the delta (Δ) configuration. The star side is typically grounded, which helps stabilize the system and manage unbalanced loads. This transformer configuration is particularly beneficial for applications where isolation, voltage transformation, and reduced phase voltage are required.

This report focuses on the mathematical model of a generalized three-phase Y – Δ transformer model.

Applications The Star-Delta configuration finds application in various domains, including:

- **Power Distribution:** Transforming voltages for distribution to various loads.
- **Motor Start Applications:** Providing reduced starting current for induction motors, which can lead to a smoother start.
- **Harmonic Mitigation:** Managing harmonics in the electrical network through proper winding connections.

Mathematical Modeling The Y – Δ transformer with turn ratio $n = 1$ is depicted in Fig. 3.7. Using the same notation, it can be seen that: The primary side phase voltages for the star configuration are:

$$v_{pa} = v_{aN1}, \quad v_{pb} = v_{bN1}, \quad v_{pc} = v_{cN1}, \quad (3.9)$$

where v_{aN}, v_{bN}, v_{cN} represent the balanced three-phase voltages with respect to ground. The secondary side (star) voltages also correspond to the line voltages, and it can be seen that:

$$\begin{bmatrix} v_{sa} \\ v_{sb} \\ v_{sc} \end{bmatrix} = \begin{bmatrix} v_{sa}^\Delta \\ v_{sb}^\Delta \\ v_{sc}^\Delta \end{bmatrix} = \sqrt{3}I \begin{bmatrix} v_{pa} \\ v_{pb} \\ v_{pc} \end{bmatrix}. \quad (3.10)$$

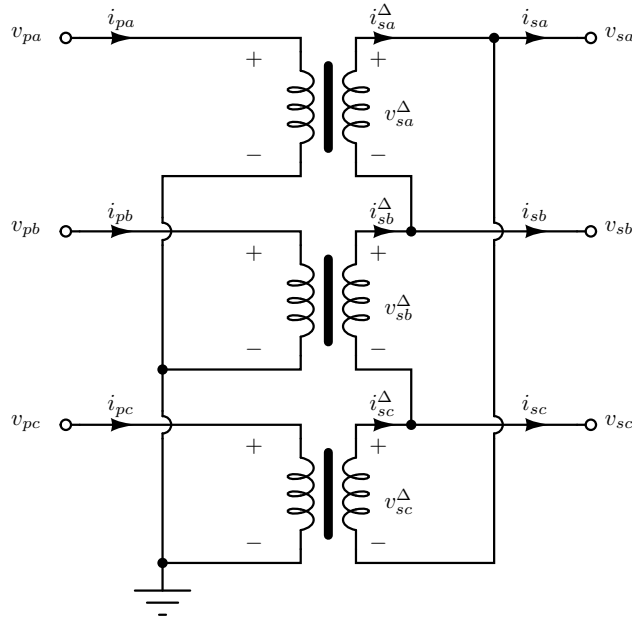


Figure 3.7: Y – Δ transformer with turn ratio $n = 1$.

Furthermore, on the star (primary) side, the three-phase balanced phase currents are i_{pa} , i_{pb} and i_{pc} . The relationship to the secondary side is as follows [7]:

$$\begin{bmatrix} i_{sa} \\ i_{sb} \\ i_{sc} \end{bmatrix} = \begin{bmatrix} 1 & 0 & -1 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} i_{sa}^\Delta \\ i_{sb}^\Delta \\ i_{sc}^\Delta \end{bmatrix} = \underbrace{\frac{1}{\sqrt{3}} \begin{bmatrix} 1 & 0 & -1 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix}}_M \begin{bmatrix} i_{pa} \\ i_{pb} \\ i_{pc} \end{bmatrix}. \quad (3.11)$$

In order to describe the transformer with a classic or real model, it is easy to represent it as a series connection of the YY transformer with Y parameters as given in equation (3.8) and the transformer model given in Fig. 3.7, as can be seen in Fig. 3.8. Here, the following relationships can be found:

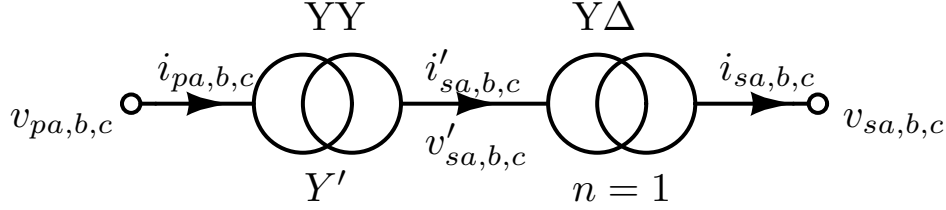


Figure 3.8: Y – Δ transformer with different models.

$$v_{sa,b,c} = \sqrt{3} I_{3 \times 3} v'_{sa,b,c}, \quad (3.12a)$$

$$v'_{sa,b,c} = \frac{1}{\sqrt{3}} I_{3 \times 3} v_{sa,b,c} \quad (3.12b)$$

$$i_{sa,b,c} = M i'_{sa,b,c}. \quad (3.12c)$$

Y Matrix Knowing that the following relation stands for YY transformer as given in equation (3.8):

$$\begin{bmatrix} i_{pa} \\ i_{pb} \\ i_{pc} \\ i'_{sa} \\ i'_{sb} \\ i'_{sc} \end{bmatrix} = Y' \begin{bmatrix} v_{pa} \\ v_{pb} \\ v_{pc} \\ v'_{sa} \\ v'_{sb} \\ v'_{sc} \end{bmatrix}, \quad (3.13)$$

then the Y matrix for a three-phase Y – Δ transformer can be derived as:

$$\begin{bmatrix} i_{pa} \\ i_{pb} \\ i_{pc} \\ i_{sa} \\ i_{sb} \\ i_{sc} \end{bmatrix} = \begin{bmatrix} I_{3 \times 3} & 0 \\ 0 & M \end{bmatrix} Y' \begin{bmatrix} v_{pa} \\ v_{pb} \\ v_{pc} \\ v'_{sa} \\ v'_{sb} \\ v'_{sc} \end{bmatrix} = \begin{bmatrix} I_{3 \times 3} & 0 \\ 0 & M \end{bmatrix} Y' \begin{bmatrix} I_{3 \times 3} & 0 \\ 0 & \frac{1}{\sqrt{3}} I_{3 \times 3} \end{bmatrix} \begin{bmatrix} v_{pa} \\ v_{pb} \\ v_{pc} \\ v_{sa} \\ v_{sb} \\ v_{sc} \end{bmatrix}, \quad (3.14)$$

which gives Y parameters as:

$$Y = \begin{bmatrix} I_{3 \times 3} & 0 \\ 0 & M \end{bmatrix} Y' \begin{bmatrix} I_{3 \times 3} & 0 \\ 0 & \frac{1}{\sqrt{3}} I_{3 \times 3} \end{bmatrix}. \quad (3.15)$$

Code Explanation The ‘TransformerYDelta’ class in the file ‘transformer_Y_Delta.cpp’ defines the behavior of the transformer. This class inherits from the base class ‘Transformer_classic’. The key responsibilities of this class are to:

- Initialize the transformer’s electrical parameters.
- Compute the Y matrix for the transformer, which consists of 6x6 elements representing the admittance relationships between the 3 primary and 3 secondary windings.
- Ensure proper resource management with a destructor.

Constructor The constructor takes parameters for the symbol, number of pins, and electrical values. It initializes the base class `Transformer_classic` with the provided values.

The Y parameters are computed using the previously calculated Y parameters for the YY transformer and then recalculated using the formula (3.15).

Destructor The destructor outputs a message indicating that the transformer object is being destroyed.

This destructor ensures proper cleanup of the object. The `TransformerYDelta` class is a robust implementation that accurately models the Y-Delta transformer, considering the essential electrical parameters and interactions. This implementation can be utilized in various electrical circuit analyses and simulations.

$\Delta - \Delta$ transformer

Introduction Three-phase Δ - Δ transformers are widely used in industrial and commercial power systems. They provide several advantages, including high power handling capability and robust performance in balancing loads across phases. It must be noted that the mutual impedance and cross-couplings between the phases are neglected in this model.

The Δ - Δ configuration offers the benefit of reduced harmonic distortion and improved voltage regulation. This configuration is commonly found in applications where substantial power is required, such as:

- **Industrial Applications:** Power distribution in manufacturing plants, motor drives, and heavy machinery operations.
- **Commercial Buildings:** Supply to large commercial facilities, office complexes, and shopping malls.
- **Power Generation:** Interfacing with generators and renewable energy systems, such as wind and solar farms.

The possible ratings for Δ - Δ transformers typically range from several kilovolt-amperes (kVA) to several megavolt-amperes (MVA), with common ratings including:

- **Small Ratings:** 50 kVA to 500 kVA for small industrial and commercial applications.
- **Medium Ratings:** 500 kVA to 5 MVA for larger industrial loads.
- **Large Ratings:** Above 5 MVA for utility-scale power distribution and large industrial processes.

Mathematical Modeling The modeling of a three-phase Δ - Δ transformer can be tackled similarly as for the $Y - \Delta$ transformer by analyzing connections in Fig. 3.9. Namely, following the same approach as before, the Y parameters are estimated as:

$$Y = \begin{bmatrix} M & 0 \\ 0 & M \end{bmatrix} Y' \frac{1}{\sqrt{3}} I_{6 \times 6}. \quad (3.16)$$

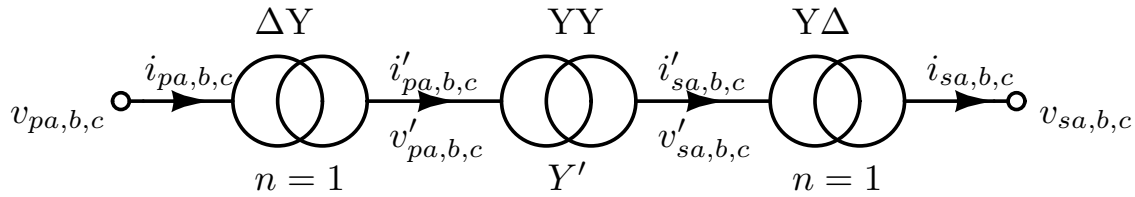


Figure 3.9: $\Delta - \Delta$ transformer with different transformer models.

Code Explanation This class models the electrical characteristics of a Δ - Δ transformer and provides methods to retrieve its parameters for analysis and simulations. It is designed the same way as for the previously described $Y - \Delta$ transformer.

$\Delta - Y$ transformer

Introduction Three-phase Δ -Y transformers are commonly used in power systems for voltage transformation and isolation between different parts of the network. The Y (or star) side typically has its neutral grounded, providing a stable reference voltage. Delta-Y transformers are preferred for many applications due to their ability to handle unbalanced loads and suppress certain harmonics.

This report generalizes the model for a three-phase Δ -Y transformer by including:

- A phase shift factor between the primary (delta) and secondary (Y) windings.
- Mutual and cross-coupling between phases for a realistic representation.

Thus it extends on the three-phase $\Delta - \Delta$ transformer model and incorporates the mutual and the cross-couplings between the three phases.

Applications The Δ -Y configuration is common in:

- **Distribution Networks:** Voltage transformation for supplying loads at lower voltage levels.
- **Power Generation:** Interfacing generators to the grid, especially when grounding is important.
- **Industrial Power Systems:** Serving factory motor loads and equipment.

Mathematical Modeling The modeling of a three-phase $\Delta - Y$ transformer can be tackled similarly as for the $Y - \Delta$ transformer by analyzing connections in Fig. 3.10. Namely, following the same approach as before, the Y parameters are estimated as:

$$Y = \begin{bmatrix} M & 0 \\ 0 & I_{3 \times 3} \end{bmatrix} Y' \begin{bmatrix} \frac{1}{\sqrt{3}} I_{3 \times 3} & 0 \\ 0 & I_{3 \times 3} \end{bmatrix}. \quad (3.17)$$

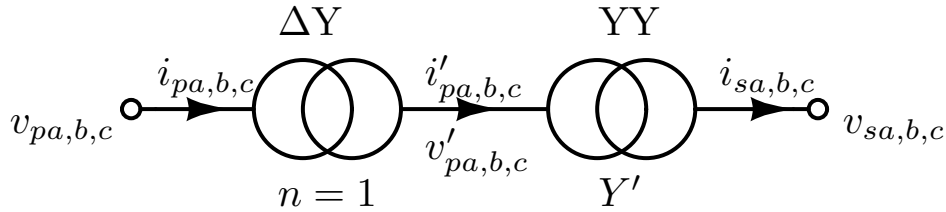


Figure 3.10: $\Delta - Y$ transformer with different transformer models.

Code Explanation This class models the electrical characteristics of a Δ - Δ transformer and provides methods to retrieve its parameters for analysis and simulations. It is designed the same way as for the previously described $Y - \Delta$ transformer.

3.2.7 Generator

Introduction In power system analysis, synchronous generators are a fundamental component of energy generation. To model the interaction between the generator and the rest of the power network, admittance parameters (Y-parameters) are often used. The Y-parameters provide a relationship between the voltages and currents at the generator's terminals and can be used in steady-state and dynamic analysis. This report presents the mathematical formulation of the Y-parameters for a synchronous generator, considering key electrical parameters such as field resistance, field inductance, direct-axis reactance, and field time constant.

Generator Model Parameters The generator's behavior can be described using the following key parameters, as depicted in Fig. 3.11:

- R_f : Field winding resistance
- L_f : Field winding inductance
- X_d : Direct-axis reactance
- T_f : Field winding time constant

These parameters represent the electrical characteristics of the synchronous machine and are used to derive the generator's impedance and admittance.

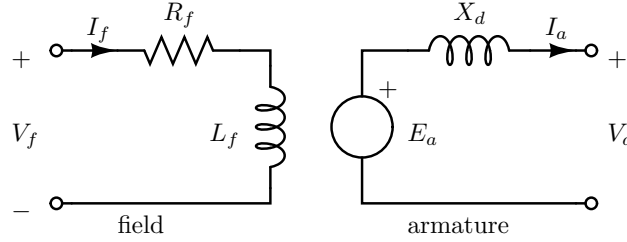


Figure 3.11: Equivalent generator model.

Mathematical Modeling In the frequency domain, the impedance of the generator can be described by its field impedance and direct-axis reactance.

The field winding presents both resistive and inductive characteristics. The impedance of the field is given by:

$$Z_f = R_f + j\omega L_f. \quad (3.18)$$

The inductive component $j\omega L_f$ represents the reactance introduced by the inductance of the field winding at a given frequency. The admittance corresponding to the field impedance is $Y_f = \frac{1}{Z_f} = \frac{1}{R_f + j\omega L_f}$. So we can write the relationship:

$$V_f = Z_f I_f. \quad (3.19)$$

The time dynamics of the generator's field winding are represented by a transfer function based on the field time constant T_f , from the exciter as depicted in Fig. x. The field transfer function is expressed as:

$$H_f = \frac{1}{1 + T_f j\omega}. \quad (3.20)$$

This transfer function is crucial for modeling the dynamic behavior of the field winding in the frequency domain. It characterizes the response of the field current to changes in the applied voltage over time.

The synchronous reactance along the direct axis of the machine is represented as a purely reactive impedance:

$$Z_d = jX_d, \quad (3.21)$$

where X_d is the direct-axis synchronous reactance.

The admittance is the reciprocal of impedance, and it provides a direct relationship between the current through and voltage across the generator terminals.

Y Parameters The generator's behavior is captured by a 2-port Y-parameter matrix, which describes the relationship between the currents and voltages at the generator's terminals. The Y-parameter matrix for the generator is given by $\mathbf{Y} = \begin{bmatrix} Y_{11} & Y_{12} \\ Y_{21} & Y_{22} \end{bmatrix}$.

The self-admittance Y_{11} is the admittance seen at the generator's field winding, which is given by the field admittance:

$$Y_{11} = Y_f = \frac{1}{R_f + j\omega L_f}$$

This term represents the ability of the field winding to conduct current for a given applied voltage.

The mutual admittance Y_{12} represents the coupling between the field winding and the direct-axis impedance. It is derived as:

$$Y_{12} = -\frac{H_f}{Z_d} = -\frac{1}{(T_f j\omega + 1)jX_d}$$

This negative value accounts for the phase relationship between the field and direct-axis circuits.

The reverse mutual admittance, Y_{21} , describes the coupling in the opposite direction, from the direct-axis circuit to the field winding. For simplicity, in many generator models, this term is often approximated as zero:

$$Y_{21} = 0$$

The self-admittance Y_{22} describes the admittance in the direct-axis circuit. This is represented as:

$$Y_{22} = -\frac{1}{Z_d} = -\frac{1}{jX_d}$$

This term indicates the relationship between the current and voltage in the direct axis of the generator.

The Y-parameter matrix provides a compact and efficient method for analyzing the steady-state and dynamic behavior of a synchronous generator. By modeling the field winding impedance, direct-axis reactance, and the field transfer function, we can accurately describe the generator's response to electrical inputs. The self-admittances Y_{11} and Y_{22} provide insight into the field and direct-axis characteristics, while the mutual admittances capture the interaction between the two. This Y-parameter model is essential for understanding the performance of the generator in power system studies.

Code Explanation This report provides a comprehensive explanation of the C++ code implemented for modelling the synchronous generator in the file `Generator.cpp` and the `Generator.h`. The synchronous generator is modelled as an electrical element with specified parameters such as field winding resistance, field winding reactance, direct-axis reactance, and the field winding time constant. For simplicity, we have neglected the governor and excitation system models within the synchronous generator and have also assumed that the mutual and cross-couplings between the phases are relatively small and hence can be neglected while formulating the Y matrices as shown in this report.

This report explains the structure and functionality of the `generator.h` file, which defines a `Generator` class used in the modeling of electrical generators. The `Generator` class is derived from the base class `Element` and includes parameters specific to the field and direct-axis dynamics of a generator. This header file is an essential part of building a generator's model in a larger electrical system simulation.

Private Members The private section contains four key parameters that describe the electrical characteristics of the generator. These parameters are assigned default values that can be overridden during object initialization.

These parameters are:

- **R_f**: Field resistance of the generator with a default value of 1.0.
- **L_f**: Field inductance, which defaults to 0.01 H.
- **X_d**: Direct-axis reactance, with a default value of 1.0.
- **T_f**: Time constant of the field winding, with a default value of 0.1 seconds.

These default values are chosen to represent typical behavior for generator components in the absence of specific data, but they can be customized during object construction.

Constructor The constructor initializes a generator object with a specific **symbol**, a number of **pins**, and a **values** vector that contains the electrical parameters. The symbol likely refers to the label used for the generator in the system, and the pins represent the number of electrical terminals.

The constructor takes three arguments:

- **symbol**: A string representing the name or label of the generator.
- **pins**: An integer representing the number of pins (or terminals) the generator has in the electrical system.
- **values**: A vector of doubles representing the parameters of the generator, which should correspond to **R_f**, **L_f**, **X_d**, and **T_f**.

When the constructor is invoked, it is expected that the values provided in the **values** vector will overwrite the default parameter values. If the size of the **values** vector is not exactly 4, an exception is thrown to ensure proper initialization.

The next part of the code defines symbolic variables for the complex frequency, $j\omega$, and converts the parameters into SymEngine symbolic types. The parameters **R_f**, **L_f**, **X_d**, and **T_f** are converted to SymEngine's **real_double** type for further symbolic manipulation. Next, the field impedance **Z_f**, admittance **Y_f**, and direct-axis impedance **Z_d** are computed. Then, the Y-parameters are based on the field admittance, direct-axis impedance, and time constant function.

Finally, the calculated Y-parameters are populated into the generator's Y matrix. The matrix is populated for all the terminals (pins) of the generator. It must be noted here that as the mutual and the cross couplings between the phases are neglected, hence the Y matrix for the three-phase case is defined as a diagonal one as shown here.

Destructor The destructor is defined as a simple empty function. As the class does not manage any dynamic memory (i.e., it does not use pointers), there is no need for custom cleanup logic in the destructor.

3.2.8 Transmission Lines

Long transmission lines are typically modeled using distributed parameters rather than lumped elements due to the significant effect of the line's length on voltage and current distribution. The telegrapher's equations describe the voltage and current along the transmission line as functions of time and space, taking into account resistance (R), inductance (L), capacitance (C), and conductance (G) per unit length. This report aims to derive the Y parameters for a long transmission line, which relates the input voltage and current to the output voltage and current, using the distributed parameter model [8].

1. **Telegrapher's Equations:** The telegrapher's equations are partial differential equations that describe the voltage $V(x, t)$ and current $I(x, t)$ at a point x along a transmission line. They are derived based on Kirchhoff's voltage and current laws, combined with the lumped element model for small segments of the line. The telegrapher's equations in the frequency domain are given as:

$$\frac{dV(x)}{dx} = -(R + j\omega L)I(x) \quad (3.22)$$

$$\frac{dI(x)}{dx} = -(G + j\omega C)V(x) \quad (3.23)$$

where:

- R is the resistance per unit length (Ω/m),
 - L is the inductance per unit length (H/m),
 - G is the conductance per unit length (S/m),
 - C is the capacitance per unit length (F/m),
 - ω is the angular frequency of the source.
2. **Characteristic Impedance and Propagation Constant:** From the telegrapher's equations, we can derive two important parameters:

- **Characteristic Impedance (Z_0):**

$$Z_0 = \sqrt{\frac{R + j\omega L}{G + j\omega C}} \quad (3.24)$$

- **Propagation Constant (γ):**

$$\gamma = \sqrt{(R + j\omega L)(G + j\omega C)} \quad (3.25)$$

The characteristic impedance describes the ratio of voltage to current for a wave traveling along the transmission line, while the propagation constant describes how the wave attenuates and changes phase as it propagates along the line.

3. **Voltage and Current Distribution Along the Line:** The general solutions for the voltage and current along the transmission line are:

$$V(x) = V_+ e^{-\gamma x} + V_- e^{\gamma x} \quad (3.26)$$

$$I(x) = \frac{V_+}{Z_0} e^{-\gamma x} - \frac{V_-}{Z_0} e^{\gamma x} \quad (3.27)$$

where V_+ and V_- represent the forward and backward traveling voltage waves, respectively. These constants can be determined by applying the boundary conditions at the input and output of the line.

4. Boundary Conditions: At $x = 0$ (the input or source end of the transmission line), the boundary conditions are:

$$V(0) = V_s \quad (3.28)$$

$$I(0) = \frac{V_s - V(0)}{Z_s} \quad (3.29)$$

where V_s is the source voltage and Z_s is the source impedance.

At $x = l$ (the load end of the transmission line), the boundary conditions are:

$$V(l) = V_+ e^{-\gamma l} + V_- e^{\gamma l} \quad (3.30)$$

$$I(l) = \frac{V(l)}{Z_L} \quad (3.31)$$

where Z_L is the load impedance.

5. Computation of Y Parameters: The Y parameters relate the input and output voltages and currents as follows:

$$\begin{bmatrix} I_1 \\ I_2 \end{bmatrix} = \begin{bmatrix} Y_{11} & Y_{12} \\ Y_{21} & Y_{22} \end{bmatrix} \begin{bmatrix} V_1 \\ V_2 \end{bmatrix}. \quad (3.32)$$

We compute the Y parameters using the voltage and current expressions at both ends of the transmission line ($x = 0$ and $x = l$):

$$I_1 = \frac{V_+}{Z_0} - \frac{V_-}{Z_0}, \quad (3.33a)$$

$$I_2 = \frac{V_+}{Z_0} e^{-\gamma l} - \frac{V_-}{Z_0} e^{\gamma l}. \quad (3.33b)$$

From these expressions, we derive the following Y parameters:

$$Y_{11} = \frac{1}{Z_0} \frac{\cosh(\gamma l)}{\sinh(\gamma l)} = \frac{1}{Z_0 \tanh(\gamma l)}, \quad (3.34a)$$

$$Y_{12} = \frac{-1}{Z_0} \frac{1}{\sinh(\gamma l)}, \quad (3.34b)$$

$$Y_{21} = Y_{12}, \quad (3.34c)$$

$$Y_{22} = Y_{11}. \quad (3.34d)$$

These Y parameters describe the transmission line's behavior regarding its input and output voltages and currents.

In this report, we derived the Y parameters for a long transmission line using the telegrapher's equations. We first computed the characteristic impedance and propagation constant, which describe how voltage and current waves propagate along the line. Using the boundary conditions at the source and load ends, we then solved for the traveling wave coefficients. We computed the Y parameters, which are crucial for analyzing transmission lines in networked power systems.

Code Explanation This report provides a comprehensive explanation of the C++ code implemented for modeling a transmission line in the file `transmissionline.cpp` and the `transmissionline.h`. The transmission line is modeled as a long line incorporating series resistance and series inductance per unit length and the shunt conductance and the susceptance per unit length.

The ‘TransmissionLine.h’ file provides a clear and structured way to define a transmission line in the context of electrical simulations. By inheriting from the ‘Element’ class, it maintains consistency within the overall framework, allowing for straightforward integration with other circuit elements. The parameters defined in this class can be utilized for further computations, such as calculating admittance or impedance in circuit analysis. Furthermore, the ‘TransmissionLine.cpp’ file contains the implementation of the ‘TransmissionLine’ class, which models the behavior of transmission lines in electrical circuit simulations. The class extends the ‘Element’ base class, initializing key parameters related to the transmission line, computing its admittance parameters, and storing them in a matrix format.

Private Member Variables The following private member variables represent the transmission line parameters, initialized with default values:

- `R_t1` = 0.01: Represents the resistance per unit length of the transmission line in ohms per meter (Ω/m).
- `L_t1` = $2.5\text{e-}7$: Represents the inductance per unit length of the transmission line in henries per meter (H/m).
- `G_t1` = $1\text{e-}9$: Represents the conductance per unit length of the transmission line in siemens per meter (S/m).
- `C_t1` = $1\text{e-}11$: Represents the capacitance per unit length of the transmission line in farads per meter (F/m).
- `length` = 1000: Represents the length of the transmission line in meters (m).

Constructor The constructor of the ‘TransmissionLine’ class initializes its parameters based on the provided values. It validates the size of the input vector to ensure it contains the correct number of parameters.

The constructor takes three parameters:

- `symbol`: A string representing the symbol of the transmission line.
- `pins`: An integer indicating the number of pins.
- `values`: A vector containing five double values representing the parameters of the transmission line.

The constructor checks also the size of the ‘values’ vector. If the size is not equal to 5, it throws an exception. Otherwise, it assigns the values to the respective member variables. The resistance, inductance, conductance, and capacitance values are converted to a specific type using ‘real_double’, which presumably returns a representation suitable for further calculations.

Next, the constructor computes intermediate values based on the angular frequency, inductance, and capacitance. The characteristic impedance Z_0 and propagation constant γ are computed using the previously defined parameters. Subsequently, the length-dependent parameters are computed.

The admittance parameters Y_{11} , Y_{12} , Y_{21} , and Y_{22} are calculated based on the previously computed values. The computed Y-parameters are stored in the ‘Y_matrix’.

Destructor The destructor is defined as a trivial operation, indicating that no special cleanup is required when the object is destroyed.

3.2.9 Overhead line

Based on realizations of the transmission line as defined in PSCAD, see Fig. 3.12, five possible realizations are defined as:

1. flat (horizontal, which presents a flat configuration without ground wires),
2. vertical,
3. delta (for at least three phases),
4. offset (for at least three phases),
5. concentric (for at least three phases).

Besides, the conductor positions can be added manually as absolute (x, y) positions. Thus, the simulator enables the creation of overhead lines with the **conductors** having the properties [9] presented in the table 11. **Ground wires** have the properties presented in table 12.

Table 11: OHL conductor parameters

Symbol	Meaning
n_b	number of bundles (or a number of phases)
n_{sb}	number of subconductors per bundle
y_{bc}	height of the lowest bundle above ground
Δy_{bc}	vertical offset between bundles
Δx_{bc}	horizontal offset between the lowest bundles
$\Delta \tilde{x}_{bc}$	horizontal offset in the case of concentric and offset organization
d_{sb}	distance between closest subconductors with equidistant concentric organization (symmetric)
d_{sag}	maximal sag offset
r_c	radius of the conductor
R_{dc}	DC resistance of the conductor
g_c	shunt conductance of the conductor
$\mu_{r,c}$	relative permeability of the conductor
positions	added manually
organization	can be flat, vertical, concentric, delta and offset

Table 12: OHL groundwires' properties

Symbol	Meaning
n_g	number of ground wires
Δx_g	relative horizontal distance between ground wires
Δy_g	relative vertical between ground wires and the lowest conductors
r_g	radius of the ground wire
$d_{g,sag}$	ground wire sag
$R_{g,dc}$	DC resistance of the ground wires
$\mu_{r,g}$	relative permeability of the ground wire

The transmission line model is constructed using the procedure from [9, ?]. The overhead transmission line consists of n_b including sub-conductors, stranding, etc. and n_g ground wires.

Each line/conductor positioned as x_c relatively starting from the central tower position and y_c vertically, measured from ground, with the sag at the midpoint between towers d_{sag} , see Fig. 3.13a. Thus, the modified vertical position is used in calculations as $\hat{y}_c = y_c - \frac{2}{3} d_{sag}$. The conductor is formed using n_{sb} sub-conductors grouped in the bundle, where all sub-conductors are grouped using a symmetrical equidistant

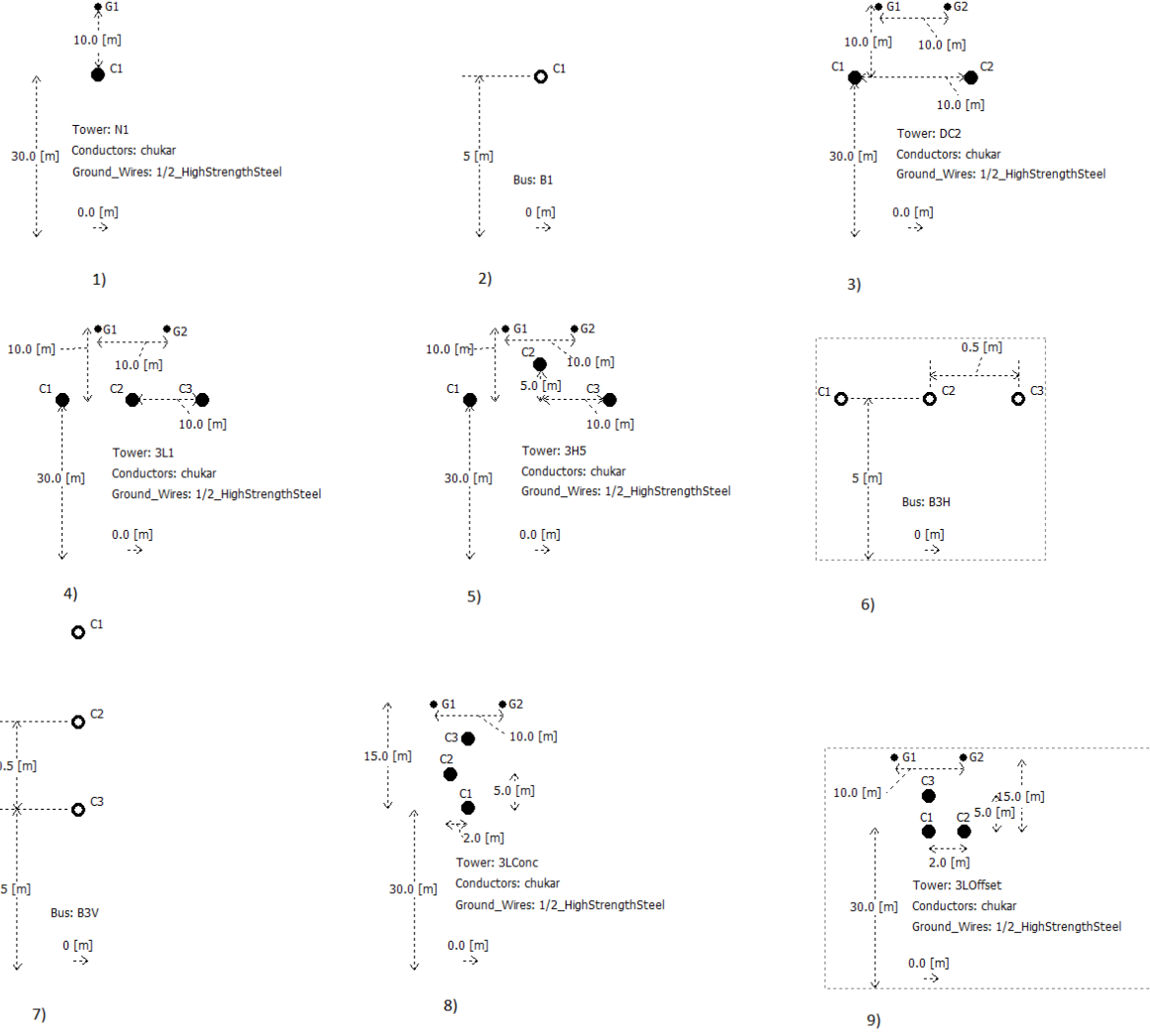


Figure 3.12: PSCAD overhead line organization: 1) single conductor with groundwire; 2) single conductor; 3) 2 conductors flat; 4) 3 conductors flat; 5) 3 conductors delta; 6) 3 conductors horizontal; 7) 3 conductors vertical; 8) 3 conductors concentric; 9) 3 conductors offset.

pattern with the distance between the two nearest sub-conductors being d_{bc} , or a bundle spacing. Using conductor position, the position of the each sub-conductor can be estimated. Knowing the angle between two sub-conductors on the circle and its radius

$$\varphi = \frac{360^\circ}{n_{sb}},$$

$$r = \frac{d_{sb}}{2 \sin(\varphi/2)}, \quad (3.35)$$

the position can be estimated starting from the angle $\varphi_s = \frac{\pi}{2}$ if the number of sub-conductors is odd, or

from $\varphi_s = \frac{\pi + \varphi}{2}$ for an even number of sub-conductors, as follows:

$$\begin{aligned} x_{bc} &= x_c + r \cos(\varphi_s + k\varphi), \\ y_{bc} &= y_c + r \sin(\varphi_s + k\varphi) - \frac{2}{3} d_{sag}, \end{aligned} \quad (3.36)$$

for $k \in \{1, 2, \dots, n_{bc}\}$. If the number of sub-conductors is equal to one, its position is given by $(x_c, \hat{y}_c)$. Each conductor is characterized by the relative permeability of the material μ_r , the conductor dc resistance R_{dc} , and the radius r_i .

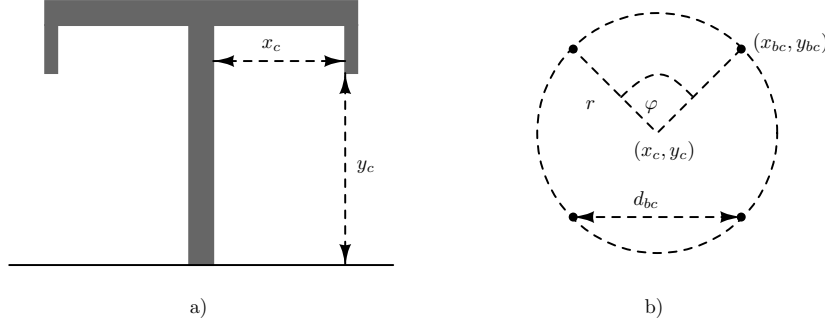


Figure 3.13: Overhead line modeling: a) tower and relative conductor positions; b) sub-conductor bundle.

Ground wires are modeled similarly, represented with their relative position (x_g, y_g) , radius r_g , dc resistance R_{gdc} , and relative permeability of the material μ_r .

Earth parameters are given with permeability μ_e , permittivity ϵ_e and conductivity ρ_e .

To represent the transmission line using ABCD parameters, it is necessary to calculate series impedance and shunt admittance matrices [9]. Both matrices are of the size $n \times n$, where $n = \sum_{i=1}^{n_c} n_{bc}^i + n_g$. The impedance matrix has the following form:

$$\mathbf{Z} = \text{diag}(\mathbf{Z}_i) + \begin{bmatrix} Z_{0,11} & \cdots & Z_{0,1n} \\ \vdots & \ddots & \vdots \\ Z_{0,n1} & \cdots & Z_{0,nn} \end{bmatrix} \quad (3.37)$$

where $Z_i = \frac{m\rho_i}{2\pi r_i} \coth(0.733mr_i) + \frac{0.3179\rho_i}{\pi r_i^2}$ for the i -th conductor/sub-conductor/ground wire and r_i is its radius, resistivity $\rho_i = R_{dc}^i \pi r_i^2$ and $m = \sqrt{j\omega \frac{\mu_0 \mu_r}{\rho_i}}$; The components $Z_{0,ij} = \frac{j\omega \mu_0}{2\pi} \log \left(\frac{\hat{D}_{ij}}{d_{ij}} \right)$ for

$$\begin{aligned} d_{ij} &= \begin{cases} \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}, & i \neq j, \\ r_i, & i = j, \end{cases} \\ D_{ij} &= \begin{cases} \sqrt{(x_i - x_j)^2 + (y_i + y_j)^2}, & i \neq j, \\ 2y_i, & i = j, \end{cases} \\ \hat{D}_{ij} &= \sqrt{(y_i + y_j + 2d_e)^2 + (x_i - x_j)^2}, \\ d_e &= \sqrt{\frac{1}{j\omega \mu_e (\sigma_e + j\omega \epsilon_e)}}. \end{aligned} \quad (3.38)$$

The shunt admittance is a matrix formed as

$$\mathbf{Y} = s\mathbf{P}^{-1} + \mathbf{G} \quad (3.39)$$

from matrix $\mathbf{P}$ with its components $\mathbf{P}_{ij} = \frac{1}{2\pi\epsilon_0} \log \left(\frac{D_{ij}}{d_{ij}} \right)$ and $\mathbf{G} = \text{diag}\{g_c\}$.

Y parameters The ABCD parameters of a transmission line can be defined as [10, 11]

$$\begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{C} & \mathbf{D} \end{bmatrix} = \begin{bmatrix} \cosh(\Gamma l) & \mathbf{Y}_c^{-1} \sinh(\Gamma l) \\ \mathbf{Y}_c \sinh(\Gamma l) & \cosh(\Gamma l) \end{bmatrix} \quad (3.40)$$

where $\Gamma = \sqrt{\mathbf{Z}\mathbf{Y}}$ and $\mathbf{Y}_c = \mathbf{Z}^{-1} \Gamma$, and l standing for the line or cable length. The formula used is based on the frequency-dependent phase domain model.

From them Y parameters calculate as follows [12]:

$$\mathbf{Y} = \begin{bmatrix} \mathbf{Y}_c \coth(\Gamma l) & -\mathbf{Y}_c \operatorname{cosech}(\Gamma l) \\ -\mathbf{Y}_c \operatorname{cosech}(\Gamma l) & \mathbf{Y}_c \coth(\Gamma l) \end{bmatrix}. \quad (3.41)$$

Code Explanation This report comprehensively explains the C++ code implemented to model an overhead transmission line in the files `OverheadLine.cpp` and `OverheadLine.h`. Class is designed to model the parameters of an overhead transmission line, which includes defining the characteristics of conductors, ground wires, and the earth. The constructor and destructor are crucial in setting up the necessary parameters for the simulation and managing the memory for dynamic objects. The overhead line is modeled as a complex structure that includes conductors and ground wires, with specific physical and electrical properties such as series resistance, series inductance, shunt conductance, and susceptance per unit length. The earth parameters, including relative permeability, permittivity, and resistivity, are also considered in the model.

The `OverheadLine.h` file defines a clear and structured way to represent an overhead transmission line for electrical simulations. By inheriting from the `Element` class, it maintains consistency within the overall framework, facilitating seamless integration with other circuit elements. The file encapsulates essential parameters, such as conductor and ground wire configurations, and attributes related to the transmission line's geometry and electrical characteristics. These parameters are essential for further computations, such as calculating impedance and admittance for circuit analysis and simulations. Additionally, the nested `Conductors` and `Groundwires` classes in this header allow for detailed configuration and positioning of the conductors and ground wires, which are crucial for the transmission line's electrical performance.

The `OverheadLine.cpp` file contains the implementation of the `OverheadLine` class. This file defines the core functionality of the class, which models the behavior of overhead transmission lines in electrical circuit simulations. It extends the `Element` base class, initializing key parameters related to the transmission line, computing its admittance and impedance parameters, and storing them in matrix form. The file also handles the estimation of conductor positions, such as flat, vertical, concentric, and offset configurations, and implements the method for calculating the admittance matrix for the transmission line. The constructor and destructor manage memory allocation for conductor and ground wire objects and ensure proper resource management.

Private Members The private member variables of the `OverheadLine` class define the internal attributes and configuration of the overhead transmission line. These variables are crucial for modeling the physical structure and electrical properties of the transmission line in simulations.

- **Conductors Class:** This nested class defines the properties of the conductors in the overhead transmission line. It includes:
 - `number_bundles` and `number_conductors_bundle` which specify the number of bundles and the number of subconductors per bundle.
 - `ybc`, `deltaYbc`, and `deltaXbc` represent the height above the ground and offsets of the bundles.
 - `dsag` and `dsb` describe the sag and spacing between subconductors.
 - `rc`, `Rdc`, and `gc` represent the conductor radius, DC resistance, and shunt conductance, respectively.
 - `positions` stores the coordinates for the positions of the conductors.
 - `organization` specifies the configuration of the conductor arrangement (e.g., flat, vertical, concentric).

- **Groundwires Class:** This nested class defines the properties of the groundwires associated with the transmission line, which are typically used for safety and performance purposes:
 - `ng` denotes the number of groundwires, usually 0 or 2.
 - `deltaXg` and `deltaYg` describe the horizontal and vertical offsets between the groundwires and the lowest conductor.
 - `rg`, `dgsag`, and `Rgdc` define the radius, sag, and DC resistance for the groundwires.
 - `positions` stores the coordinates for the positions of the groundwires.

- **Earth Parameters:** The `earthParameters` tuple holds the earth-related parameters, including the relative permeability μ_r , relative permittivity ϵ_r , and resistivity ρ , which are essential for the accurate modeling of the transmission line's interaction with the earth.

- **Length:** The `length` variable specifies the length of the transmission line, which is essential for computing the impedance and admittance parameters.

- **Matrices:** The `P`, `Y`, and `Z` matrices are used for storing various electrical parameters related to the overhead line. `P` is a matrix that holds additional data, while `Y` and `Z` are Dense matrices representing the admittance and impedance matrices of the line.

- **Eliminate Flag:** The `eliminate` boolean flag is used to control the inclusion or elimination of certain parameters in the calculations, depending on the specific requirements of the simulation.

- **Organization Coordinates:** The `organization_x_values` and `organization_y_values` vectors store the coordinates of the conductor positions for different organizational schemes (e.g., flat, vertical).

These private members are the building blocks for the overhead transmission line model, enabling detailed simulations and analyses of its behavior in an electrical network.

Constructor The constructor of the `Overhead_Line` class takes several parameters that initialize the properties of the line, such as the conductor properties, ground wire properties, earth parameters, and line length. The constructor also initializes the nested `Conductors` and `Groundwires` classes to define the properties of these elements.

```
1 Overhead_Line(const std::string& symbol, double length, std::tuple<double, double, double> earth,
2               std::tuple<std::string, std::vector<int>, std::vector<double>, double, double, double, double>
3               conductor,
4               std::tuple<int, std::vector<double>, double> groundwire);
```

The constructor accepts the following parameters:

- **symbol (string):** A symbolic identifier for the overhead line, used to differentiate between different lines or types of lines.
- **length (double):** Specifies the length of the transmission line in kilometers.
- **earth (tuple):** This tuple contains parameters for the earth, which are the relative permeability (μ_r), relative permittivity (ϵ_r), and resistivity (ρ) of the earth.
- **conductor (tuple):** This tuple holds various properties of the conductors, such as their organization type, the number of bundles and subconductors, and the dimensions of the conductors.
- **groundwire (tuple):** This tuple provides information about the ground wires, such as the number of ground wires, their spacing, and other relevant properties.

The constructor initializes the `length`, `earthParameters`, and dynamically allocates memory for the `conductors` and `groundwires` objects, calling their respective constructors. It also sets the default values for other attributes and prepares the line for further computation of electrical parameters.

Destructor The `Overhead_Line` class includes a destructor to clean up the dynamically allocated memory used by the `Conductors` and `Groundwires` objects. The destructor ensures that when an instance of the `Overhead_Line` class is destroyed, the memory allocated for these nested objects is properly released.

```
1 ~Overhead_Line();
```

This destructor ensures the proper deallocation of memory, preventing memory leaks in the application. Specifically, it deletes the `conductors` and `groundwires` objects that were dynamically allocated in the constructor.

The destructor is automatically invoked when the object goes out of scope or when it is explicitly deleted, and it is essential for ensuring the efficient memory management of the `Overhead_Line` object in simulations involving multiple transmission lines.

3.2.10 Cable

The cable groups are implemented by focusing on the configurations of the cables available in PSCAD, where a cable can be included either inside the pipe, called pipe-type cables, or placed underground. Cables are usually coaxial with up to 4 layers of both conductors and insulators.

Cables can be insulated or pipe-type coaxial cables. At the moment, only a group of coaxial cables is implemented in the package. A cable group consists of n cables, each with a maximum of three conducting layers and three insulation layers, as seen in Fig. 3.14. The conducting layers of the cable are denoted as the core, sheath, and armor. There are insulators Between the conducting layers, except for the last conductor, where the insulator is not a strict necessity but is common. For each conductor, the following set of parameters is given: r_i^c and r_o^c as conductor's inner and outer radius in meters, conductor relative permeability μ_r^c and conductor resistivity ρ_c (in $[\Omega\text{m}]$). The insulator is described using the following parameters: r_i^i and r_o^i are the insulator's inner and outer radius in meters, ϵ^i is the insulator relative permittivity and μ_r^i the insulator relative permeability.

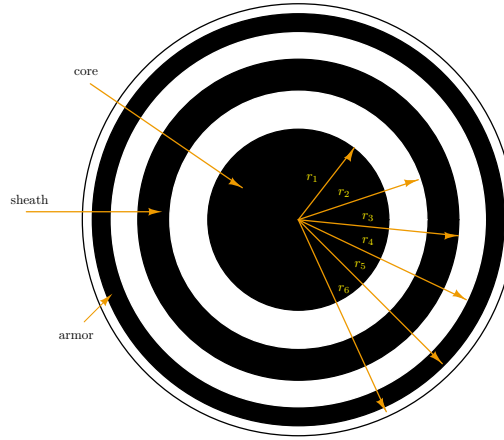


Figure 3.14: Coaxial cable.

The configuration parameters can also be modified by adding two semiconducting layers in Insulator 1 and implementing the sheath consisting of the wire screen and outer sheath layer. The procedure described in [5] is applied in that case.

- Conductor surface impedance

A hollow conductor surface impedance is given by:

$$\begin{aligned} Z_{aa} &= \frac{\rho_c m}{2\pi r_i^c} \coth(m(r_o^c - r_i^c)) + \frac{\rho^c}{2\pi r_i^c (r_i^c + r_o^c)} \left[\frac{\Omega}{\text{m}} \right] \quad \text{for inner surface,} \\ Z_{bb} &= \frac{\rho_c m}{2\pi r_o^c} \coth(m(r_o^c - r_i^c)) + \frac{\rho^c}{2\pi r_o^c (r_i^c + r_o^c)} \left[\frac{\Omega}{\text{m}} \right] \quad \text{for outer surface,} \\ Z_{ab} &= \frac{\rho_c m}{\pi(r_i^c + r_o^c)} \operatorname{csch}(m(r_o^c - r_i^c)) \left[\frac{\Omega}{\text{m}} \right], \end{aligned} \quad (3.42)$$

where $m = \sqrt{j\omega\mu_r^c}$ For a non-hollow conductor, the outer surface impedance is

$$Z_{bb} = \frac{\rho_c m}{2\pi r_o^c} \coth(0.733mr_o^c) + \frac{0.3179\rho^c}{\pi r_o^{c2}} \left[\frac{\Omega}{\text{m}} \right]. \quad (3.43)$$

- The insulator layer between two conductors has an impedance

$$Z_i = \frac{j\omega\mu_0\mu_r^i}{2\pi} \log\left(\frac{r_o^i}{r_i^i}\right). \quad (3.44)$$

- The earth return impedance of the cable and mutual between cables is

$$Z_g = \frac{j\omega \mu_g}{2\pi} \left(-\log \left(\frac{\gamma m D}{2} \right) + \frac{1}{2} - \frac{2}{3} m H \right), \quad (3.45)$$

for

$$D = \begin{cases} \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} & \text{for cables } i \neq j, \\ r_i & \text{radius of the cable } i, \end{cases}$$

$$H = \begin{cases} y_i + y_j & \text{for cables } i \neq j, \\ 2y_i & \text{for the cable } i, \end{cases} \quad (3.46)$$

and $\gamma \approx 0.5772156649$ being Euler's constant.

According to [13, 14], one cable is represented with its series impedance $\mathbf{Z}_{ii}$ matrix. Each matrix $\mathbf{Z}_{ii}$ has the size $n_c \times n_c$ and its entries for $j \in \{1, \dots, n_c - 1\}$ are given by

$$\begin{aligned} \mathbf{Z}_{ii} \langle j, j \rangle &= Z_{bb}^j + Z_i^j + Z_{aa}^{j+1}, \\ \mathbf{Z}_{ii} \langle j, j+1 \rangle &= Z_{ii} \langle j+1, j \rangle = -Z_{ab}^{j+1}, \\ \mathbf{Z}_{ii} \langle n_c, n_c \rangle &= Z_{bb}^{n_c} + Z_i^{n_c} + Z_g^{ii}, \end{aligned} \quad (3.47)$$

and otherwise, the matrix entries are 0.

Mutual surface impedances between the cables are given by matrix $\mathbf{Z}_{ij}$ having all components equal to Z_g^{ij} .

The shunt admittance matrix can be estimated as $\mathbf{Y} = s\mathbf{P}^{-1}$ and matrix P , which has the form

$$\mathbf{P} = \begin{bmatrix} \mathbf{P}_{11} & \mathbf{P}_{12} & \cdots & \mathbf{P}_{1n} \\ \vdots & \ddots & & \vdots \\ \mathbf{P}_{n1} & \mathbf{P}_{n2} & \cdots & \mathbf{P}_{nn} \end{bmatrix}.$$

Matrices $\mathbf{P}_{ii}$ have components

$$\mathbf{P}_{ii} = \begin{bmatrix} P_c + P_s + P_a & P_s + P_a & P_a \\ P_s + P_a & P_s + P_a & P_a \\ P_a & P_a & P_a \end{bmatrix} + \begin{bmatrix} P_{ii} & P_{ii} & P_{ii} \\ P_{ii} & P_{ii} & P_{ii} \\ P_{ii} & P_{ii} & P_{ii} \end{bmatrix}, \quad (3.48)$$

where $P_{c,s,a}$ belong respectively to core, shield and armor insulators and have the following values: $P = \frac{\log(r_o/r_i)}{2\pi\epsilon}$ and $P_{ii} = \frac{\log(2h_i/r)}{2\pi\epsilon_0}$ is an earth return. Matrices $\mathbf{P}_{ij}$, for $i \neq j$, have all components equal to $P_{ij} = \frac{\log(D_2/D_1)}{2\pi\epsilon_0}$, where $D_1 = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ and $D_2 = \sqrt{(x_i - x_j)^2 + (y_i + y_j)^2}$ [13].

It is valid to assume that sheath and armor are grounded, so Kron reduction is allowed. Using Kron reduction, as proposed in [15, 16], a compact shunt admittance and series impedance model is obtained when applied to the matrices $\mathbf{Y}$ and $\mathbf{Z}$.

Y parameters are calculated the same as in the equation (3.41).

Code Explanation The **Cable** class models the parameters of an electrical cable, including conductors, insulators, positions, and other characteristics essential for electrical simulations. It is derived from the **Element** class, inheriting its general properties while introducing more specific elements for cable representation. The **Cable** class includes two nested classes: **Conductor** and **Insulator**. The **Conductor** class represents the conducting layer of the cable, while the **Insulator** class models the insulating layer. The class provides constructors, member functions, and accessors for manipulating the properties of the cable, including the addition of conductors, insulators, and positions. It also includes functionality for setting and retrieving the earth parameters, type, and configuration and computing the $\mathbf{Y}$ parameters for electrical analysis.

Private Members The private member variables include:

- **length** (double): The cable length in meters.
- **conductors** (std::map): A map that stores the conductors, where the key is a symbolic name and the value is a pointer to a **Conductor** object.
- **insulators** (std::map): A map that stores the insulators, where the key is a symbolic name, and the value is a pointer to an **Insulator** object.
- **positions** (std::vector): A vector of pairs representing the positions of cable elements in a 2D space.
- **earth_parameters** (std::tuple): A tuple storing the earth parameters, including relative permeability, relative permittivity, and resistivity.
- **configuration** (std::string): The configuration type of the cable (e.g., coaxial).
- **type** (std::string): The type of cable (e.g., underground).
- **eliminate** (bool): A flag indicating whether the cable should be eliminated from computations.
- **Z, Y** (DenseMatrix): Matrices used for storing impedance and admittance data.
- **P** (MatrixXd): A matrix related to the power or other properties of the cable.

Constructor The **Cable** class has two constructors:

- **Default Constructor:** The default constructor initializes the cable with a length of 0, type as "underground", and configuration as "coaxial". It also initializes the **eliminate** flag to **true**.
- **Parameterized Constructor:** This constructor takes parameters for the cable symbol, pins, type, configuration, length, earth parameters, conductors, insulators, and positions. It initializes all these attributes and prepares the cable object for simulation use.

Destructor The destructor is responsible for cleaning up any dynamically allocated memory associated with the cable's conductors and insulators. It ensures that all resources are properly released when the object is destroyed.

```
1 ~Cable();
```

This ensures that the dynamically allocated memory for conductors and insulators is freed when the cable object is no longer in use, preventing memory leaks.

3.2.11 MMC

The Modular Multilevel Converter (MMC) is an emerging technology in modern high-voltage direct current (HVDC) systems, renewable energy integration, and motor drives. MMCs provide high efficiency, scalability, and reduced harmonic distortion compared to traditional voltage-source converters (VSCs). Due to their modular structure, MMCs can operate at high power/voltage ratings while maintaining low switching losses and improved output voltage quality.

MMC Architecture An MMC consists of multiple cascaded submodules (SMs) per phase, typically arranged in an upper and lower arm configuration. Each phase of the converter consists of:

- **Submodules (SMs):** Each SM contains capacitors and semiconductor switches, enabling independent voltage control.
- **Arms:** Each phase-leg has an upper and lower arm, controlling the output voltage and current.
- **Arm Inductors:** Limiting circulating currents and improving system stability.
- **DC-Link:** Connects the converter to the DC grid, providing the required power exchange.

The general structure of a three-phase MMC is shown in Figure 3.15.

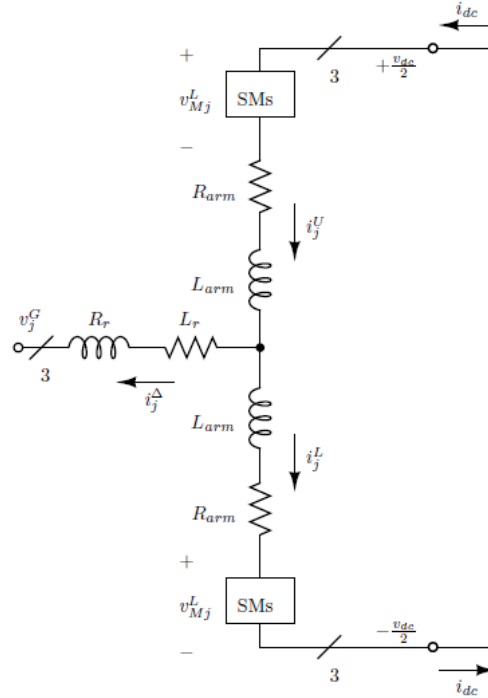


Figure 3.15: General structure of a three-phase MMC.

Operating Principles The MMC operates based on the selective activation of submodules, generating a staircase output voltage waveform. The main operational principles include:

- **Capacitor Voltage Balancing:** Ensuring uniform charge distribution across submodules.
- **Circulating Current Suppression:** Reducing unwanted AC and DC currents flowing within the converter's arms that do not contribute to power transfer between the AC and DC sides that improve efficiency.

- **Modulation Techniques:** Various pulse-width modulation (PWM) methods, such as Nearest Level Control (NLC) and Carrier-Based PWM, are used to improve the output waveform quality.

MMC Model To model the MMC dynamics, we define the upper and lower arm voltages and currents as follows:

$$v_{su,l}^k = m_{u,l}^k v_{cu,l}^k, \quad i_{su,l}^k = m_{u,l}^k i_{u,l}^k \quad (3.49)$$

Where $m_{u,l}^k$ are the insertion indices for the upper and lower arms, and $v_{u,l}^k$ are the capacitor voltages. Using the summation (Σ) and difference (Δ) notation, we define:

$$i^{\Delta k} = i_u^k - i_l^k, \quad i^{\Sigma k} = \frac{i_u^k + i_l^k}{2}, \quad (3.50)$$

$$v_c^{\Delta k} = \frac{v_{cu}^k - v_{cl}^k}{2}, \quad v_c^{\Sigma k} = \frac{v_{cu}^k + v_{cl}^k}{2}, \quad (3.51)$$

$$m^{\Delta k} = m_u^k - m_l^k, \quad m^{\Sigma k} = m_u^k + m_l^k, \quad (3.52)$$

$$v_s^{\Delta k} = \frac{-v_{su}^k + v_{sl}^k}{2} = \frac{-m^{\Delta k} v_c^{\Sigma k} + m^{\Sigma k} v_c^{\Delta k}}{2}, \quad (3.53)$$

$$v_s^{\Sigma k} = \frac{v_{su}^k + v_{sl}^k}{2} = \frac{m^{\Sigma k} v_c^{\Sigma k} + m^{\Delta k} v_c^{\Delta k}}{2}. \quad (3.54)$$

The resulting differential equations governing the MMC dynamics in the dq reference frame are:

$$\dot{i}^{\Delta dq} = \frac{v_s^{\Delta dq} - (\omega L_t J_2 + R_t I_2) i^{\Delta dq} - v_{pcc}^{dq}}{L_t}, \quad (3.55)$$

$$\dot{i}^{\Sigma dq} = \frac{-v_s^{\Sigma dq} + (R_a I_2 - 2\omega L_a J_2) i^{\Sigma dq}}{L_a}, \quad (3.56)$$

$$\dot{i}_z^{\Sigma} = \frac{v_{dc}}{2L_a} - \frac{v_s^{\Sigma z} + R_a i^{\Sigma z}}{L_a}, \quad (3.57)$$

$$\dot{v}_c^{\Delta dq} = \frac{N}{2C} i_s^{\Delta dq} - \omega J_2 v_c^{\Delta dq}, \quad (3.58)$$

$$\dot{v}_c^{\Delta Z dq} = -\frac{N}{8C} \Psi - 3\omega J_2 v_c^{\Delta Z dq}, \quad (3.59)$$

$$\dot{v}_c^{\Sigma dqz} = \frac{N}{2C} i_s^{\Sigma dqz} + 2\omega J_3 v_c^{\Sigma dqz}, \quad (3.60)$$

$$\dot{v}_{dc} = \frac{1}{C_{dc}} (i_{dc} - 3i^{\Sigma z}). \quad (3.61)$$

The following parameters are used in the model:

- N = number of submodules
- C = submodule capacitance
- R_a = arm resistance
- L_a = arm inductance
- $R_t = R_r + \frac{R_a}{2}$ = equivalent AC resistance
- $L_t = L_r + \frac{L_a}{2}$ = equivalent AC inductance
- C_{dc} = equivalent DC side capacitance

Additional equations are introduced to refine the MMC model further and capture the effects of modulation and frequency variations. These terms account for modulation indices, power balance, and the interaction of the submodule voltages with system parameters:

$$i_s^{\Delta dq} = P_\omega(t) \left(P_{-2\omega}^{-1}(t) m_{\tau_d}^{\Sigma dqz} \circ \frac{P_\omega^{-1}(t) i^{\Delta dqz}}{2} + P_\omega^{-1}(t) m_{\tau_d}^{\Delta dqZ} \circ P_{-2\omega}^{-1}(t) i^{\Sigma dqz} \right), \quad (3.62)$$

$$i_s^{\Sigma dq} = P_{-2\omega}(t) \left(P_{-2\omega}^{-1}(t) m_{\tau_d}^{\Sigma dqz} \circ P_{-2\omega}^{-1}(t) i^{\Sigma dqz} + P_\omega^{-1}(t) m_{\tau_d}^{\Delta dqZ} \circ P_\omega^{-1}(t) \frac{i^{\Delta dqz}}{2} \right), \quad (3.63)$$

$$v_s^{\Sigma dq} = -\frac{P_\omega}{2}(t) \left(P_\omega^{-1}(t) m_{\tau_d}^{\Sigma dqz} \circ P_{-2\omega}^{-1}(t) v_c^{\Sigma dqz} + P_{-2\omega}^{-1}(t) m_{\tau_d}^{\Sigma dqZ} \circ P_\omega^{-1}(t) v_c^{\Delta dqz} \right), \quad (3.64)$$

$$v_s^\Sigma = \frac{P_{-2\omega}}{2}(t) \left(P_\omega^{-1}(t) m_{\tau_d}^{\Delta dqz} \circ P_\omega^{-1}(t) v_c^{\Delta dqz} + P_{-2\omega}^{-1}(t) m_{\tau_d}^{\Sigma dqZ} \circ P_{-2\omega}^{-1}(t) v_c^{\Sigma dqz} \right), \quad (3.65)$$

$$\Psi = \begin{bmatrix} i^{\Delta d} m_{\tau_d}^{\Sigma d} + 2i^{\Sigma d} m_{\tau_d}^{\Delta d} + i^{\Delta q} m_{\tau_d}^{\Sigma q} + 2i^{\Sigma q} m_{\tau_d}^{\Delta q} + 4i^{\Sigma z} m_{\tau_d}^{\Delta Zd} \\ i^{\Delta q} m_{\tau_d}^{\Sigma d} + 2i^{\Sigma d} m_{\tau_d}^{\Delta q} + i^{\Delta d} m_{\tau_d}^{\Sigma q} + 2i^{\Sigma q} m_{\tau_d}^{\Delta d} + 4i^{\Sigma z} m_{\tau_d}^{\Delta Zq} \end{bmatrix} \quad (3.66)$$

The above model provides a comprehensive framework for controlling and simulating MMC-based HVDC systems, enabling efficient power transfer with minimized losses and improved system stability.

The above differential equations describe the state-space representation of an MMC operating in the dqz reference frame. These equations facilitate the analysis of dynamic behavior and the design of control strategies, enabling optimized power flow and stability in AC/DC power systems. Properly tuning insertion indices and balancing arm currents and capacitor voltages are essential for maintaining stable operation and reducing circulating currents.

The insertion indices, denoted by m , are obtained as follows:

$$m = \begin{bmatrix} m^{\Delta dq} & m^{\Delta Zdq} & m^{\Sigma dqz} \end{bmatrix}^T = \frac{2}{v_{dc}} \begin{bmatrix} -v_{s,ref}^{\Delta dqZ} \\ v_{s,ref}^{\Sigma dqz} \end{bmatrix} \quad (3.67)$$

Where:

- $m^{\Delta dq}$, $m^{\Delta Zdq}$, and $m^{\Sigma dqz}$ are the components of the insertion indices.
- v_{dc} is the DC voltage.
- $v_{s,ref}^{\Delta dqZ}$ and $v_{s,ref}^{\Sigma dqz}$ are the reference voltages for the respective components.

Computational time delays in the control architecture are included by considering the time-delayed versions of the insertion indices. The time-delayed version of m , denoted by m_{τ_d} , is expressed as:

$$m_{\tau_d} = m(t - \tau_d) = \begin{bmatrix} m_{\tau_d}^{\Delta dq} \\ m_{\tau_d}^{\Delta Zdq} \\ m_{\tau_d}^{\Sigma dqz} \end{bmatrix} = \begin{bmatrix} m^{\Delta dq}(t - \tau_d) \\ m^{\Delta Zdq}(t - \tau_d) \\ m^{\Sigma dqz}(t - \tau_d) \end{bmatrix} \quad (3.68)$$

Where τ_d represents the time delay.

Note: Except for $v_{c,ref}^z$, which is set to $\frac{v_{dc}}{2}$ by default, the other voltage references are set to zero unless specified by a particular controller.

The above equations define the insertion indices m , which are crucial for modeling the system's behavior. These indices are expressed as a vector containing three components: $m^{\Delta dq}$, $m^{\Delta Zdq}$, and $m^{\Sigma dqz}$. These components represent various system parameters, such as voltage differences and impedance references. The indices are scaled by $\frac{v_{dc}}{2}$, where v_{dc} is the DC voltage of the system. The voltage reference values $v_{s,ref}^{\Delta dqZ}$ and $v_{s,ref}^{\Sigma dqz}$ are used in the computation of these insertion indices.

The model also accounts for **time delays** in the control architecture, essential for simulating realistic control systems. These time delays are introduced by considering delayed versions of the insertion indices. The time-delayed version of m , denoted by m_{τ_d} , is expressed as a time-shifted version of m , with a time

shift of τ_d , which is the time delay. This allows the model to consider the effect of delays in the system's response.

A key point to note is the voltage reference $v_{zc,ref}$, which is set to $\frac{v_{dc}}{2}$ by default. All other voltage references are set to zero unless a specific controller explicitly defines them. This is a simplifying assumption, ensuring that the model is easy to handle unless specific control actions are specified.

The time delay in the system and its approximation are described. The pure time delay is represented as:

$$T_{\text{delay}} = e^{-s\tau_d} \quad (3.69)$$

Where s is the Laplace operator and τ_d is the time delay.

The Pade approximation of the time delay is given by:

$$T_{\text{delay}} = e^{-s\tau_d} \approx \frac{b_0 + b_1(\tau_d s)^1 + \dots + b_m(\tau_d s)^m}{a_0 + a_1(\tau_d s)^1 + \dots + a_n(\tau_d s)^n} \quad (3.70)$$

Where $b_0, b_1, \dots, b_m$ and $a_0, a_1, \dots, a_n$ are coefficients determined by the approximation order.

In state-space form, the time-delayed system is described by the following equations:

$$\dot{x}_{\tau_d}(t) = A_{\tau_d}x_{\tau_d}(t) + B_{\tau_d}u_{\tau_d}(t) \quad (3.71)$$

$$y_{\tau_d}(t) = C_{\tau_d}x_{\tau_d}(t) + D_{\tau_d}u_{\tau_d}(t) \quad (3.72)$$

Where:

$$u_{\tau_d}(t) = [m^{\Delta dq} \quad m^{\Sigma dqz}]^T \quad \text{and} \quad y_{\tau_d}(t) = [m_{\tau_d}^{\Delta dqz} \quad m_{\tau_d}^{\Sigma dqz}]^T \quad (3.73)$$

Here, $x_{\tau_d}(t)$ is the state vector, $u_{\tau_d}(t)$ is the input vector, and $y_{\tau_d}(t)$ is the output vector, with appropriate matrices A_{τ_d} , B_{τ_d} , C_{τ_d} , and D_{τ_d} for the system.

Code Explanation The `MMC` class models a Modular Multilevel Converter (MMC) and inherits from the `Element` base class. It defines the electrical characteristics of the converter, including active and reactive power, AC and DC voltages, arm inductance, resistance, and capacitance. The class also supports the integration of controllers and filters to enhance system performance. The MMC provides methods for initializing and managing controllers and filters. MMC supports adding controllers and filters, which are stored in a structured manner.

Key functionalities include:

- **Initialization of MMC Parameters:** Stores electrical and physical characteristics such as voltage, current, impedance, and the number of submodules.
- **Controller and Filter Integration:** Supports adding controllers and filters dynamically.
- **Diagnostic Functions:** Includes methods to print MMC parameters and their sub-elements.

Private Members The `MMC` class maintains private member variables that define the convertethconverter's properties and sub-elements.

- **Electrical Characteristics:**

- `omega_0`: Nominal system frequency.
- `P`, `Q`, `P_dc`: Active, reactive, and DC power.
- `P_min`, `P_max`, `Q_min`, `Q_max`: Power operation limits.
- `theta`: AC voltage angle.
- `V_m`, `V_dc`: AC and DC voltages.
- `L_arm`, `R_arm`, `C_arm`: Arm inductance, resistance, and capacitance.

- **N**: Number of submodules.
- **L_reactor**, **R_reactor**: Reactor inductance and resistance.
- **t_delay**: Time delay parameter.

- **Controller and Filter Management:**

- **subElements**: Vector of sub-elements (controllers/filters).
- **controls**: Map of controllers integrated into the MMC.
- **filters**: Map of filters connected to the MMC.

Constructor The constructor initializes all MMC parameters, ensuring proper integration into a power system simulation. It assigns default values to electrical variables and allows dynamic initialization with user-defined values.

```

1 // Constructor with parameters
2 MMC(const std::string& symbol, int inputPins, int outputPins,
3     double omega, double activePower, double reactivePower, double dcPower,
4     double minActivePower, double maxActivePower, double minReactivePower,
5     double maxReactivePower, double angle, double acVoltage, double dcVoltage,
6     double armInductance, double armResistance, double armCapacitance,
7     int numSubmodules, double reactorInductance, double reactorResistance,
8     double timeDelay)
9 : Element(symbol, inputPins, outputPins),
10   omega_0(omega), P(activePower), Q(reactivePower), P_dc(dcPower),
11   P_min(minActivePower), P_max(maxActivePower), Q_min(minReactivePower),
12   Q_max(maxReactivePower), theta(angle), V_m(acVoltage), V_dc(dcVoltage),
13   L_arm(armInductance), R_arm(armResistance), C_arm(armCapacitance),
14   N(numSubmodules), L_reactor(reactorInductance),
15   R_reactor(reactorResistance), t_delay(timeDelay) {}

```

This constructor initializes:

- MMC power flow parameters (**P**, **Q**, **V_dc**).
- Arm circuit properties (**L_arm**, **R_arm**, **C_arm**).
- Reactor elements (**L_reactor**, **R_reactor**).
- Time delay constraints (**t_delay**).

Destructor The destructor ensures proper memory deallocation for MMC components.

```

1 ~MMC() {}

```

Since all dynamically allocated elements (controllers and filters) are stored using smart pointers, explicit memory deallocation is unnecessary. The destructor ensures safe cleanup when an MMC instance is destroyed.

Controller and Filter Integration in MMC Modular Multilevel Converters (MMC) require advanced control strategies and filtering techniques to ensure stable operation, power quality, and harmonic mitigation. The **Controller** and **Filter** classes are designed to be integrated into the MMC framework, enhancing system performance through dynamic control and signal conditioning.

Controller in MMC The **Controller** class regulates MMC operation by adjusting control signals such as voltage, current, and power setpoints. It implements a proportional-integral (PI) controller with additional parameters for damping and bandwidth regulation.

Controller Features

- **Proportional-Integral (PI) Control:** Ensures stable and accurate tracking of reference signals.
- **Damping Ratio and Bandwidth Adjustment:** Helps tuning controller performance to prevent oscillations.
- **Dynamic Parameter Updates:** Allows real-time adjustments to control gains based on system conditions.

The controller is initialized with the following constructor:

```
1 // Constructor for Controller class
2 Controller(const std::string& symbol, int pins, double Kp, double Ki,
3           double zeta, double bandwidth, const std::vector<double>& ref)
4 : Element(symbol, pins, pins), // Same number of input and output pins
5   Kp(Kp), Ki(Ki), zeta(zeta), bandwidth(bandwidth), reference(ref) {}
```

Controller Parameters The key parameters defining the controller behavior are:

- K_p : Proportional gain
- K_i : Integral gain
- ζ : Damping ratio
- B : Bandwidth (Hz)
- Reference values: A vector of setpoints used to regulate system variables

The controller supports real-time updates of its parameters using:

```
1 // Method to update controller parameters
2 void updateParameters(double newKp, double newKi, double newZeta, double newBandwidth);
```

Controller in MMC Integration The MMC integrates multiple controllers to regulate different aspects, such as:

- ****AC voltage regulation**:** Ensures steady-state voltage levels.
- ****Active and reactive power control**:** Maintains grid stability.
- ****Current loop control**:** Ensures proper current distribution among converter arms.

Filter in MMC In addition to controllers, filters are required to mitigate harmonics, reduce switching noise, and enhance waveform quality. The **Filter** class implements low-pass and damping filters within the MMC framework.

Filter Features

- **Time Constant-Based Filtering:** Smoothens transient variations in signals.
- **Damping Control:** Prevents undesired oscillations.
- **Bandwidth Configuration:** Allows tuning of filter response.

The filter is initialized using:

```
1 // Constructor for Filter class
2 Filter(const std::string& symbol, int pins, double timeConstant, double zeta, double bandwidth)
3 : Element(symbol, pins, pins), // Filters have same input and output pins
4   timeConstant(timeConstant), zeta(zeta), bandwidth(bandwidth) {}
```


Filter Parameters The filter behavior is defined by:

- T : Time constant
- ζ : Damping ratio
- B : Bandwidth (Hz)

The filter allows dynamic updates to its characteristics using:

```
1 // Method to update filter parameters
2 void updateParameters(double newTimeConstant, double newZeta, double newBandwidth);
```

Filter in MMC Integration Filters are critical in MMC operation for:

- ****Suppressing harmonics**** generated by switching devices.
- ****Improving signal quality**** by removing noise from control loops.
- ****Stabilizing system response**** by damping high-frequency oscillations.

Controller and Filter Management in MMC The MMC class allows the dynamic addition of controllers and filters, ensuring modularity and scalability.

```
1 // Add a filter or controller to MMC
2 void addSubElement(const std::string& subElementName, const std::shared_ptr<Element>& subElement) {
3     subElements.push_back({ subElementName, subElement });
4 }
```

The integration of controllers and filters within the MMC architecture ensures robust performance by:

- Maintaining voltage and power balance.
- Suppressing unwanted harmonics and improving waveform quality.
- Enhancing system stability through dynamic tuning.

This modular approach allows flexibility in control strategies, making MMC adaptable for grid conditions and power applications.

3.2.12 Software Testing

Unit tests have been implemented to ensure the C++ code of the different power system components works as expected and produces the desired results. The tests have been written using the GoogleTest C++ testing framework in combination with the Eigen library for linear algebra. The tests focus on validating the correct initialization of the components and computation of the Y matrix. They are organized per component, such that all the tests for a particular component are located in a “test|component|.cpp” file. Inside the file, there is one test case per public function of the corresponding component class.

For example, the testimpedance.cpp file shown below defines and implements the tests for the impedance component class defined in impedance.h (see Listing 1). Given that the impedance class has no public methods other than the class constructor and destructor, the tests are limited to verifying the correct initialization of the component and computation of the Y matrix.

```
1 #include <gtest/gtest.h>
2 #include "Impedance.h"
3
4 // Test suite for Impedance class
5 class TestImpedance : public testing::Test {};
6
7 // Test cases for Impedance constructor
8 TEST_F(TestImpedance, TestConstructor) {
9     // Optional: Let GTest capture Harmony's output to std::cerr so
10     // it is not displayed in the test log
11     testing::internal::CaptureStderr();
12
13     // Test exception for invalid number of pins
14     // Pins must be > 0
15     EXPECT_THROW(Impedance("z", 0, DenseMatrix(1, 1, { div(integer(1), mul(j, omega)) })), std::
        invalid_argument);
16     EXPECT_THROW(Impedance("z", -1, DenseMatrix(1, 1, { div(integer(1), mul(j, omega)) })), std::
        invalid_argument);
17
18     // Test exception for invalid number of values
19     // values.ncols() = 1, or values.ncols() = num pins
20     // case: pins = 1 & values.ncols = 2
21     EXPECT_THROW(Impedance("z", 1, DenseMatrix(1, 2, { div(integer(1), mul(j, omega)), mul(j, omega) })), std
        ::invalid_argument);
22     // case: pins = 3 & values.ncols = 2
23     EXPECT_THROW(Impedance("z", 3, DenseMatrix(1, 2, { div(integer(1), mul(j, omega)), mul(j, omega) })), std
        ::invalid_argument);
24     // case: pins = 2 & values.ncols = 2
25     EXPECT_NO_THROW(Impedance("z", 2, DenseMatrix(1, 2, { div(integer(1), mul(j, omega)), mul(j, omega) })));
26
27     // Stop GTest capturing Harmony's output to std::cerr
28     testing::internal::GetCapturedStderr();
29 }
30
31 // Test cases for Y Matrix
32 TEST_F(TestImpedance, TestYMatrix) {
33     // Optional: Let GTest capture Harmony's output to std::cerr so
34     // it is not displayed in the test log
35     testing::internal::CaptureStderr();
36
37     // Case 1
38     Impedance z1("z1", 1, DenseMatrix(1, 1, { div(integer(1), mul(j, omega)) })); // 1/wi
39     MatrixXcd y1 = z1.compute_y_parameters_num(10000);
40     MatrixXcd y1expected(2, 2);
41     y1expected(0, 0) = std::complex<double>(0, 10000);
42     y1expected(0, 1) = std::complex<double>(0, -10000);
43     y1expected(1, 0) = std::complex<double>(0, -10000);
44     y1expected(1, 1) = std::complex<double>(0, 10000);
45     EXPECT_TRUE(y1.isApprox(y1expected, 1e-9));
46
47     // Case 2
48     Impedance z2("z2", 2, DenseMatrix(1, 2, { div(integer(1), mul(j, omega)), mul(integer(2), mul(j, omega)) }
        )); // 1/wi, 2*(wi)
```

```

49 MatrixXcd y2 = z2.compute_y_parameters_num(1500);
50 MatrixXcd y2expected(4, 4);
51 std::complex<double> i1500(0, 1500);
52 std::complex<double> i0(0, 0);
53 std::complex<double> i33(0, 0.000333333);
54 y2expected << i1500, i0, -i1500, i0,
55               i0, -i33, i0, i33,
56               -i1500, i0, i1500, i0,
57               i0, i33, i0, -i33;
58 EXPECT_TRUE(y2.isApprox(y2expected, 1e-9));
59
60 // Stop GTest capturing Harmony's output to std::cerr
61 testing::internal::GetCapturedStderr();
62 }

```

- Initialization test (lines 7-28): Creates impedance components based on different values for the number pins and impedances. The impedance constructor is called multiple times with incorrect combinations of values to verify that a C++ `std::invalid_argument` exception is thrown.
- Y matrix test (lines 31-62): Creates two impedance components based on different values of the number of pins and impedances and calculates the Y matrix for each of them. The calculated Y matrices are compared to the expected results via the `isApprox` method of the Eigen library.

There are currently 17 implemented tests: 10 tests for the element C++ class, 2 for impedance and 5 for the load. It is intended to increase the number of tests to cover the public methods of all C++ classes within Harmony.

3.3 Equivalent Impedance of the Subnetwork

The HVDC system induces harmonics in the grid. These harmonics entail serious stability issues and are a real source of problems. Before going into system modeling to study the harmonic effect, a basic overview of the nature of the generated harmonics is important—a brief but detailed overview of the nature of harmonics or the multiples of generated harmonics.

3.3.1 Harmonic Resonance Modal Analysis Approach

Y parameters can determine the impedance between input and output pins. It uses the harmonic resonance modal analysis approach [17, 18]. The input impedance can be calculated using the following steps. First, the matrix $\mathbf{M}$ is constructed as a zero quadratic matrix with the size $N \times N$. The entries that belong to the currents of the input and output nodes, denoted as k_i , $i \in \{1, \dots, N\}$, are equaled with $\mathbf{M} \langle k_i, k_i \rangle = 1$.

$$\begin{bmatrix} Y_{1,1} & Y_{1,2} & \cdots & Y_{1,k_i} & \cdots & Y_{1,N} \\ Y_{2,1} & Y_{2,2} & \cdots & Y_{2,k_i} & \cdots & Y_{2,N} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ Y_{k_i,1} & Y_{k_i,2} & \cdots & Y_{k_i,k_i} & \vdots & Y_{k_i,N} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ Y_{N,1} & Y_{N,2} & \cdots & Y_{N,k_i} & \cdots & Y_{N,N} \end{bmatrix} \begin{bmatrix} V_1 \\ V_2 \\ \vdots \\ V_k \\ \vdots \\ V_N \end{bmatrix} = \mathbf{M} \begin{bmatrix} I_1 \\ I_2 \\ \vdots \\ I_k \\ \vdots \\ I_N \end{bmatrix}, \quad (3.74)$$

The inverse of the matrix $\mathbf{Y}$, even in the case of singularity, can be determined as [18]:

$$\mathbf{Z} = \mathbf{T} \mathbf{\Lambda} \mathbf{T}^{-1},$$

where $\mathbf{T}$ presents column eigenvectors and $\mathbf{\Lambda}$ diagonal complex eigenvalues of the matrix $\mathbf{Y}$.

The last step is to determine the impedances between input and output pins as:

$$\mathbf{Z}_{eq} = \mathbf{Z} \mathbf{M}$$

and to pick only the values on the positions corresponding to the input and output nodes.

To get the impedance “visible” from the input nodes, it is necessary to perform Kron elimination of the output nodes as the last step [19]. The so-called Kron reduction [19], is often used for reducing grounded conducting layers of the overhead lines and cables. It will be formulated only for the Y parameters, but it can be applied in the same way for Z parameters.

The Y matrix is divided into four parts: the matrix part with the superscript 11 should be kept, 22 should be eliminated and 12 and 21 are the interconnections.

$$\mathbf{Y} = \begin{bmatrix} \mathbf{Y}_{11} & \mathbf{Y}_{12} \\ \mathbf{Y}_{21} & \mathbf{Y}_{22} \end{bmatrix}$$

The new Y parameters are then given by:

$$\tilde{\mathbf{Y}} = \mathbf{Y}_{11} - \mathbf{Y}_{12} \mathbf{Y}_{22}^{-1} \mathbf{Y}_{21}.$$

3.3.2 Adjusted Modified Nodal Analysis Approach

The solving procedure can be successfully applied with the following steps:

1. Determine the input and output of the circuit where you want to determine the equivalent impedance, see Fig. 3.16.
2. Assign input and output currents as depicted in Fig. 3.17.
3. Build Modified Nodal Analysis (MNA) [20] by adding all components except the ones on the direct connection to the input and output pins. Those components are considered as non-existent as seen in Fig. 3.17. Furthermore, the following sorting of variables should be applied: $[V_i, I_i, V_o, I_o, V]$, where

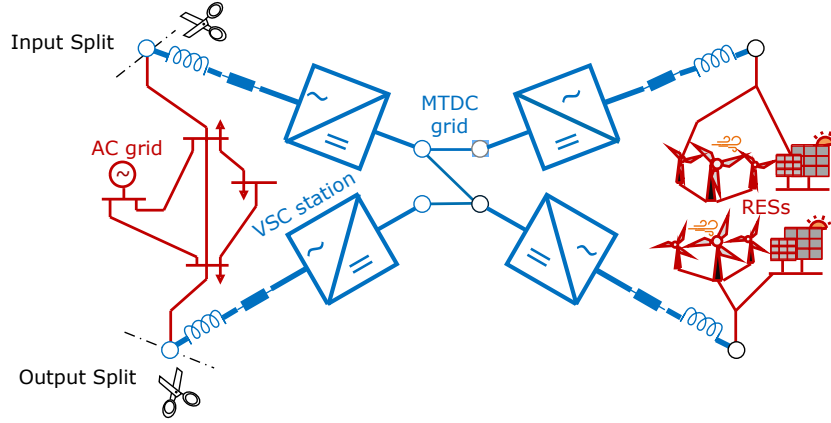


Figure 3.16: Circuit cut.

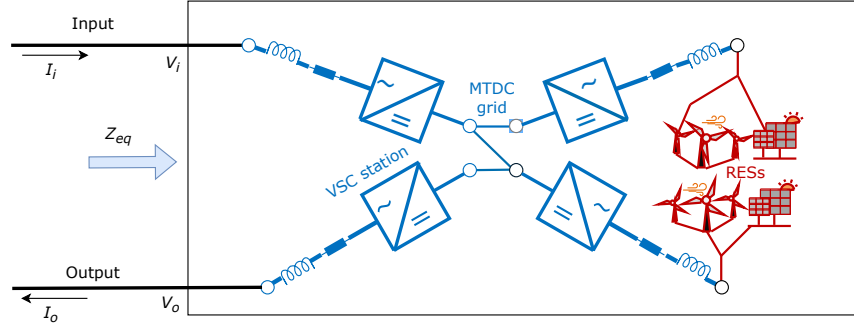


Figure 3.17: Equivalent impedance determination.

V_i and I_i present voltages and currents at the input like depicted in Fig. 3.17, V_o and I_o are output voltages and currents, and V are other voltage buses/nodes that are not input and output ones.

There are no equations written for the ground bus, as standardly defined in the MNA approach [20].

4. The formulated equations will be as follows:

$$\underbrace{\begin{bmatrix} Y_{ii} & \text{diag}\{-1\} & Y_{io} & 0 & Y_{iv} \\ \text{diag}\{1\} & 0 & 0 & 0 & 0 \\ Y_{oi} & 0 & Y_{oo} & \text{diag}\{1\} & Y_{ov} \\ 0 & 0 & \text{diag}\{1\} & 0 & 0 \\ Y_{vi} & 0 & Y_{vo} & 0 & Y_{vv} \end{bmatrix}}_M \underbrace{\begin{bmatrix} V_i \\ I_i \\ V_o \\ I_o \\ V \end{bmatrix}}_x = \underbrace{\begin{bmatrix} V_1 \\ 0 \\ V_2 \\ 0 \\ 0 \end{bmatrix}}_z, \quad (3.75)$$

where Y_{xy} are Y parameters for the specified block of nodes x and y . Values V_1 and V_2 are symbolic vectors of values assigned to input and output nodes. With 0 is denoted zero matrix. For simplicity, the dimensions are skipped in the previous equation.

5. The previous equation (3.75) is solved using the reduced row echelon form [21] on the concatenated matrices M and z . This will solve the system of equations by enforcing the values for variables x to diagonal 1 values, and on the position of z column would be the expected solution.

6. By taking values for V_i and I_i and dividing them, we obtain the equivalent impedance per phase.

References

- [1] B. Kocuk, S. S. Dey, and X. A. Sun, “Strong socp relaxations for the optimal power flow problem,” *Operations Research*, vol. 64, no. 6, pp. 1177–1196, 2016.
- [2] L. Gan and S. H. Low, “Optimal power flow in direct current networks,” in *52nd IEEE Conference on Decision and Control*, 2013, pp. 5614–5619.
- [3] J. J. Grainger, *Power system analysis*. McGraw-Hill, 1999.
- [4] W. G. Hurley and W. H. Wölfe, *Transformers and inductors for power electronics: theory, design and applications*. John Wiley & Sons, 2013.
- [5] L. Wu, “Impact of ehv/hv underground power cables on resonant grid behavior,” 2014.
- [6] A. Izadian, *Fundamentals of Modern Electric Circuit Analysis and Filter Synthesis*. Springer, 2019.
- [7] A. B. Salas, “Control interactions in power systems with multiple vsc hvdc converters,” *Ph. D. thesis*, 2018.
- [8] H. Saadat *et al.*, *Power system analysis*. McGraw-hill, 1999, vol. 2.
- [9] J. A. Martinez-Velasco, *Power system transients: parameter determination*. CRC press, 2017.
- [10] F. Castellanos and J. Marti, “Full frequency-dependent phase-domain transmission line model,” *IEEE transactions on Power Systems*, vol. 12, no. 3, pp. 1331–1339, 1997.
- [11] A. Morched, B. Gustavsen, and M. Tartibi, “A universal model for accurate calculation of electromagnetic transients on overhead lines and underground cables,” *IEEE Transactions on Power Delivery*, vol. 14, no. 3, pp. 1032–1038, 1999.
- [12] W. Leterme, “Y-parameters model of cross-bonded cables including bonding wire impedances,” *IEEE Transactions on Power Delivery*, vol. 36, no. 1, pp. 495–498, 2020.
- [13] A. Ametani, “A general formulation of impedance and admittance of cables,” *IEEE transactions on power apparatus and systems*, no. 3, pp. 902–910, 1980.
- [14] P. de Arizon and H. W. Dommel, “Computation of cable impedances based on subdivision of conductors,” *IEEE Transactions on Power Delivery*, vol. 2, no. 1, pp. 21–27, 1987.
- [15] F. De Léon, M. L. Márquez-Asensio, and G. Álvarez-Cordero, “Effects of conductor counter-transposition on the positive-sequence impedance and losses of cross-bonded cables,” *IEEE Transactions on Power Delivery*, vol. 26, no. 3, pp. 2060–2063, 2011.
- [16] R. A. Rivas and J. R. Marti, “Calculation of frequency-dependent parameters of power cables: Matrix partitioning techniques,” *IEEE transactions on power delivery*, vol. 17, no. 4, pp. 1085–1092, 2002.
- [17] H. A. Brantsæter, “Harmonic resonance mode analysis and application for offshore wind power plants,” Master’s thesis, NTNU, 2015.
- [18] W. Xu, Z. Huang, Y. Cui, and H. Wang, “Harmonic resonance mode analysis,” *IEEE Transactions on Power Delivery*, vol. 20, no. 2, pp. 1182–1190, 2005.
- [19] F. Dorfler and F. Bullo, “Kron reduction of graphs with applications to electrical networks,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 60, no. 1, pp. 150–163, 2012.
- [20] C.-W. Ho, A. Ruehli, and P. Brennan, “The modified nodal approach to network analysis,” *IEEE Transactions on Circuits and Systems*, vol. 22, no. 6, pp. 504–509, 1975.
- [21] J. Bird, *Electrical circuit theory and technology*. Routledge, 2017.